Reporte

<table>
<tr><td colspan="2">Nombres: Equipo 5

Rodrigo Noguera Pérez

Andrik Hasiel Pérez Limón

Erick Antonio Páez Estrada</td><td>Matrículas:

2136238

2118898

2120761</td></tr>
<tr><td colspan="2">Nombre del curso: Componentes de Lógica de Negocio</td><td>Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td colspan="2">Módulo Netacad: 6</td><td>Actividad: Reporte de Solución de Software Basada en Lógica de Negocio, TecnoStock (Caso 05)</td></tr>
<tr><td colspan="3">Fecha: 06/11/2026</td></tr>
<tr><td colspan="3">Bibliografía (Formato APA)

Equipo TecnoStock. (2026a). Actividad 2: Análisis y documentación de la empresa TecnoStock

Equipo TecnoStock. (2026b). Actividad 3: Modelo de dominio, arquitectura y diseño de TecnoStock

Componentes de Lógica de Negocios. (2026). Producto integrador de aprendizaje: Proyecto final. Implementación de una solución de software basada en lógica de negocio

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.</td></tr>
</table>

Reporte

Resumen ejecutivo

Este reporte documenta el Producto Integrador de Aprendizaje del Equipo 5 sobre el caso TecnoStock, una cadena de cinco tiendas de electrónica. Partimos de lo que construimos en las unidades 2 y 3 (BRD, historias de usuario, BPMN, glosario y el diseño DDD/UML) y lo bajamos a un prototipo que de verdad corre: un dominio con reglas probadas, una API REST, base de datos, interfaz web, pruebas automatizadas, Docker Compose y un workflow de GitHub Actions.

El prototipo cubre completo el ciclo de vida del apartado: reserva temporal de 15 minutos, anticipo mínimo del 20 %, confirmación con vigencia de 30 días, liquidación con ticket y código QR, entrega con número de serie único, cancelación y vencimiento con abono al monedero del cliente. También incluye la consulta de disponibilidad y el bloqueo de las unidades apartadas para que nadie más las venda. Las transferencias entre sucursales, la venta directa y las devoluciones sí quedaron diseñadas en la U3, pero **no las programamos**; lo decimos de frente en el alcance y en la sección de limitaciones.

En la ejecución final la suite reportó **41 pruebas aprobadas y 98.37 % de cobertura** sobre `src/`, contra el 80 % que pide la rúbrica. Además, corrimos los 120 casos y las 20 transiciones del paquete de datos del PIA, que tratamos como material ilustrativo y no como las reglas originales de TecnoStock.

Propósito y objetivos

Propósito. Convertir el análisis de negocio de TecnoStock en una solución de software organizada por capas, donde las reglas del negocio vivan en el dominio, se puedan probar sin base de datos y se puedan demostrar desde una API y una interfaz.

Objetivo general

Integrar los requerimientos y procesos del caso TecnoStock en una aplicación funcional basada en lógica de negocio, aplicando Domain-Driven Design, Specification, State Machine, API REST, persistencia, pruebas automatizadas, CI/CD y contenedores.

Objetivos específicos

- Relacionar las reglas del BRD con las historias de usuario, los escenarios Gherkin, el BPMN y el glosario.
- Modelar el dominio con agregados, objetos de valor, eventos y servicios coherentes con el caso.
- Exponer los casos de uso mediante una API REST documentada con OpenAPI/Swagger.

- Persistir en SQLite para trabajo local y en PostgreSQL con Docker Compose.
- Verificar el comportamiento con pruebas unitarias y de integración y mantener al menos 80 % de cobertura.
- Dejar una ejecución reproducible: variables de entorno, Docker Compose y un pipeline en GitHub Actions.

Alcance y organización del trabajo

La guía del PIA pide integrar los productos de las unidades 2 y 3, implementar DDD, Specification, State Machine y API REST, organizar el repositorio, comprobar las pruebas con cobertura mínima de 80 %, mantener CI/CD y usar Docker Compose para base de datos, aplicación y pruebas. Lo organizamos en las cuatro fases que se ven abajo.

Fase del proyecto	Propósito	Producto principal
1. Análisis del negocio	Entender el problema, las reglas y los procesos de TecnoStock.	BRD, historias con Gherkin, BPMN y glosario.
2. Arquitectura	Traducir el análisis a un modelo de software mantenible.	DDD, UML, capas, patrones, SOLID y ADRs.
3. Implementación	Construir el dominio, los casos de uso, la API y la interfaz.	Código en `src/`, base de datos, API y frontend.
4. Calidad y producción	Comprobar, automatizar y preparar la demostración.	Pruebas, cobertura, CI/CD, Docker Compose y Swagger.

Qué sí quedó implementado y qué no

Preferimos dejar clara la frontera entre lo diseñado y lo programado, porque el diseño de la U3 es más grande que el prototipo.

Implementado y probado	Diseñado en la U3, sin implementar
Consulta de disponibilidad y bloqueo de unidades apartadas. Apartado con anticipo del 20 %, reserva de 15 min y vigencia de 30 días. Liquidación con ticket QR y entrega con serie única. Cancelación, expiración y vencimiento con abono único al monedero. Specifications combinables, máquina de estados y eventos de dominio. API REST, interfaz web, pruebas, Docker Compose y CI.	Transferencias entre sucursales (US-03 a US-07). Venta directa con cobro (US-13). Devoluciones con revisión del Supervisor. Escaneo real del QR en almacén (US-15). Pasarela de pago, autenticación y roles. Scheduler permanente de vencimientos.

1. Fase de análisis del negocio

1.1 Contexto y problemática

TecnoStock vende computadoras, celulares, accesorios y componentes en cinco sucursales y permite apartar ciertos productos con un anticipo. El BRD de la U2 describe cuatro problemas que se repiten en la operación diaria, y de ahí salió casi todo el diseño:

- La existencia que muestra el sistema no coincide con el inventario físico.
- Se llega a vender un producto que ya estaba apartado para otro cliente.
- Es difícil rastrear qué número de serie se entregó a quién.
- Los criterios para aceptar devoluciones cambian de una persona a otra.

El objetivo que nos fijamos fue centralizar la disponibilidad y el control de las operaciones, proteger las unidades apartadas y mantener trazabilidad desde el apartado hasta la entrega física. Los documentos de las unidades anteriores (BRD, historias, BPMN y glosario) los usamos como base de las decisiones técnicas y no como adorno del reporte.

Actores del sistema

Actor	Responsabilidad principal
Cliente	Solicita la compra o el apartado, paga y presenta el ticket para recoger.
Vendedor	Consulta inventario, atiende al cliente e inicia solicitudes.
Cajero	Cobra, emite comprobantes y registra los pagos del apartado.
Supervisor	Autoriza excepciones, transferencias y devoluciones.
Almacén	Escanea series, prepara y recibe transferencias y registra la salida física.
Sistema	Controla disponibilidad, expiraciones, transiciones, eventos y comprobantes.

1.2 Levantamiento y reglas del negocio

Del BRD salieron cinco necesidades del negocio y un par de restricciones que mandan sobre todo lo demás: lo apartado no se vende a otra persona y todo movimiento debe poder rastrearse. El BRD también establece que no íbamos a inventar valores que el caso no trae, así que lo que no estaba definido se documentó como pendiente en lugar de suponerse.

Reporte

Necesidades de negocio (BRD, sección 5.1)

ID	Necesidad	Historias relacionadas
BR1	Consultar la disponibilidad de stock por sucursal.	US-01, US-02
BR2	Cobrar con métodos autorizados, generar comprobante y descontar inventario.	US-11, US-13, US-14
BR3	Identificar la unidad exacta mediante su número de serie.	US-15, US-16
BR4	Reservar mercancía mediante un anticipo.	US-08 a US-12
BR5	Validar de forma estructurada las devoluciones.	Pendiente (US-17 y US-18 propuestas, sin validar)

Reglas de negocio que el prototipo hace cumplir

ID	Regla	Fuente	Dónde se verifica
RN-01	El apartado exige un anticipo mínimo del 20 % del total.	US-08, US-10	`AnticipoMinimo`
RN-02	Máximo cinco artículos activos por cliente, sumando todos sus apartados.	US-08, ADR-04	`MaximoArticulosPorCliente`
RN-03	La reserva temporal dura 15 minutos; después se cancela y libera el stock.	US-08, US-09	`Apartado.expirar_reserva`
RN-04	El apartado confirmado vence a los 30 días y su anticipo se abona al monedero una sola vez.	US-10, US-12	`Apartado.vencer`, `Monedero`
RN-05	Liquidar exige cubrir exactamente el saldo y genera ticket con código QR.	US-11	`Apartado.liquidar`
RN-06	Cada unidad entregada lleva una serie única, y la salida física ocurre una sola vez.	US-16, BR3	`Apartado.entregar`
RN-07	Un producto apartado no se puede vender ni apartar de nuevo.	BRD 5.2, US-06	`Inventario.apartar`
RN-08	Si una operación falla, no queda nada a medias: se hace rollback.	US-14	`UnitOfWork`

Reglas del BRD que quedaron fuera del prototipo

- Solo el Supervisor autoriza o rechaza transferencias, y una transferencia con discrepancia no entra al inventario destino (US-04, US-07).
- Registro de venta directa y cancelación por pago declinado con una pasarela real (US-13, US-14).
- Política de devoluciones: plazo, condiciones y reembolso están sin definir en el caso, y por eso no las inventamos.

1.3 Historias de usuario y criterios Gherkin

El catálogo de la U2 tiene 16 historias de usuario agrupadas en cuatro módulos. Cada una trae su criterio de aceptación en formato Dado/Cuando/Entonces, y ese formato lo usamos para escribir las pruebas. La tabla indica además en qué estado quedó cada historia en el prototipo: **7 completas, 3 parciales y 6 que solo existen como diseño.**

ID	Rol y necesidad	Estado en el prototipo
US-01	Vendedor: consultar la disponibilidad local de un producto.	Implementada
US-02	Vendedor: consultar la disponibilidad en otras sucursales.	Parcial: la API devuelve el mapa por sucursal, pero solo cargamos una (SUC-01).
US-03	Vendedor: solicitar una transferencia de mercancía.	Solo diseño
US-04	Supervisor: autorizar o rechazar transferencias.	Solo diseño
US-05	Almacén origen: preparar y registrar el envío.	Solo diseño
US-06	Almacén destino: recibir y verificar la transferencia.	Solo diseño
US-07	Almacén destino: registrar una discrepancia en tránsito.	Solo diseño
US-08	Vendedor: crear un apartado con anticipo del 20 % y reserva de 15 min.	Implementada
US-09	Sistema: liberar la reserva temporal si no se paga a tiempo.	Implementada
US-10	Cajero: registrar el anticipo y confirmar por 30 días.	Implementada
US-11	Cajero: liquidar el saldo y generar ticket con QR.	Implementada
US-12	Sistema: vencer el apartado y mandar el anticipo al monedero.	Implementada
US-13	Cajero: registrar una venta directa con cobro.	Solo diseño
US-14	Cajero: cancelar la transacción si el pago no se aprueba.	Parcial: hay cancelación y rollback, pero no pasarela de pago.
US-15	Almacén: escanear el ticket QR para ver lo que se entrega.	Parcial: el ticket incluye el código QR como texto, sin escaneo.
US-16	Almacén: escanear series y registrar la salida física.	Implementada

Ejemplos de criterios Gherkin llevados a pruebas

ID	Escenario (Given / When / Then)	Prueba que lo cubre
US-08	**Given** un cliente que selecciona un artículo y no excede el límite de 5 **When** inicia el apartado **Then** el sistema exige al menos 20 % de anticipo y abre una reserva temporal de 15 minutos.	`test_anticipo_minimo` `test_reserva_y_anticipo_reglas`
US-09	**Given** un apartado en reserva temporal **When** pasan 15 minutos sin que se pague el anticipo **Then** el sistema cancela la operación y libera el stock.	`test_us09_reserva_expirada_libera`
US-12	**Given** un apartado confirmado que cumple 30 días sin liquidarse **When** se ejecuta la validación de tiempo **Then** se libera el stock y el anticipo se abona al monedero, una sola vez.	`test_us12_vencimiento_monedero_idempotente`

La trazabilidad completa (BRD, historia, regla, componente y prueba) está en la sección 4.6 y en `docs/architecture/trazabilidad.md` del repositorio.

1.4 Procesos BPMN y glosario de dominio

El BPMN de la U2 describe cinco carriles (Cliente, Vendedor, Cajero, Supervisor y Almacén). El proceso arranca con la solicitud de compra o apartado, consulta disponibilidad, pide una transferencia cuando falta stock local, decide si es venta o apartado, cobra, emite el ticket con QR y termina con la entrega y el registro de serie. El diagrama tiene más alcance que el prototipo; el tramo del apartado, el pago y la entrega es el que implementamos.

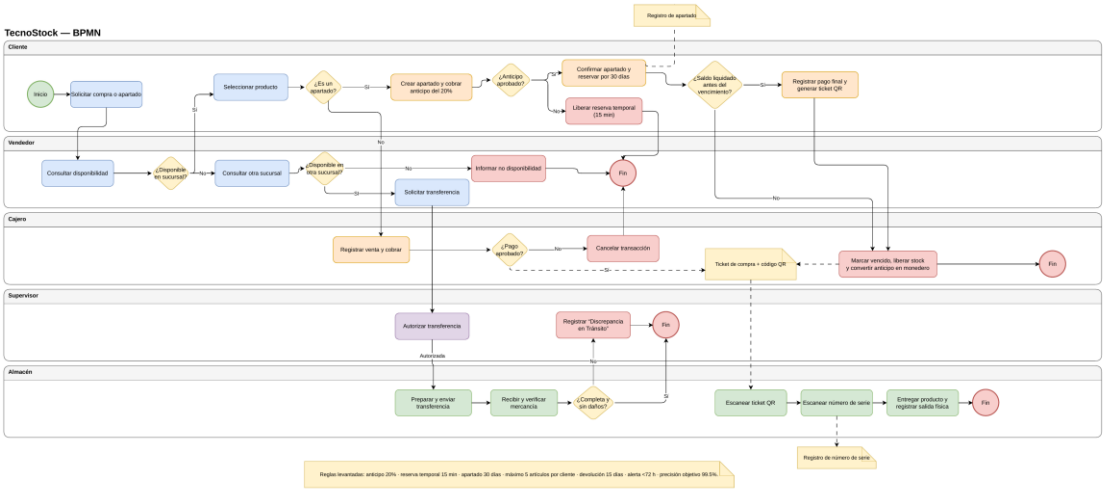


Figura 1. BPMN de la U2 con los carriles de Cliente, Vendedor, Cajero, Supervisor y Almacén.

El glosario fija el lenguaje que compartimos entre negocio y código: los nombres que aparecen aquí son los mismos que usan las clases, los eventos y los endpoints.

Término	Significado dentro del sistema
Apartado	Reserva de productos para un cliente con anticipo y vigencia.
Anticipo	Pago parcial mínimo del 20% para confirmar el apartado.
Reserva temporal	Bloqueo provisional de existencias durante 15 minutos.
Stock disponible	Existencia física menos unidades apartadas, comprometidas y reservadas para transferencia.
Ticket QR	Código que permite consultar el detalle de una venta pagada.
Unidad seriada	Pieza física identificada por un número de serie único.
Transferencia	Movimiento de mercancía entre sucursales con autorización y recepción.
Discrepancia en tránsito	Faltante o daño que detiene el ingreso al destino.
Monedero	Saldo a favor generado por un apartado confirmado vencido.
Specification	Objeto que encapsula una condición y permite combinarla con AND, OR y NOT.
Aggregate Root	Raíz que protege invariantes de un conjunto del dominio.
Unit of Work	Agrupa escrituras de un caso de uso en commit o rollback.

2. Fase de arquitectura

2.1 Diseño guiado por el dominio (DDD)

Elegimos Domain Model como patrón central porque el apartado vive en el tiempo: tiene plazos de 15 minutos y 30 días, bloquea inventario, mueve dinero y cambia de estado con reglas estrictas. Con Transaction Script o Table Module esas reglas terminarían repartidas entre endpoints y consultas, y nada garantizaría que un apartado vencido no pueda liquidarse. La decisión quedó registrada como ADR-01.

En el diseño de la U3 definimos seis raíces de agregado (Apartado, Inventario, Venta, Transferencia, Monedero y Devolución). En el prototipo programamos tres, que son las que sostienen el flujo del apartado.

Elemento DDD	Responsabilidad en TecnoStock
Apartado (agregado)	Controla estado, líneas, importes, plazos, comprobante y series. Es quien aplica la máquina de estados.
Inventario (agregado)	Existencia por sucursal y producto; aparta, libera y descuenta sin dejar cantidades negativas.
Monedero (agregado)	Saldo a favor del cliente; abona el anticipo de un apartado vencido una sola vez.
Dinero (objeto de valor)	Importes con `Decimal`, para no arrastrar errores de punto flotante.
PoliticaApartado (objeto de valor)	Parámetros del negocio (20 %, 5 artículos, 15 min, 30 días) que se inyectan, por eso las pruebas pueden cambiarlos.
Actor y producto (catálogo)	Datos de referencia que vienen de los CSV: cliente activo o preferente, precio y si el producto está habilitado.
Eventos de dominio	Hechos ya ocurridos (ReservaTemporalCreada, ApartadoVencido, etc.) que el bus reparte a los handlers.

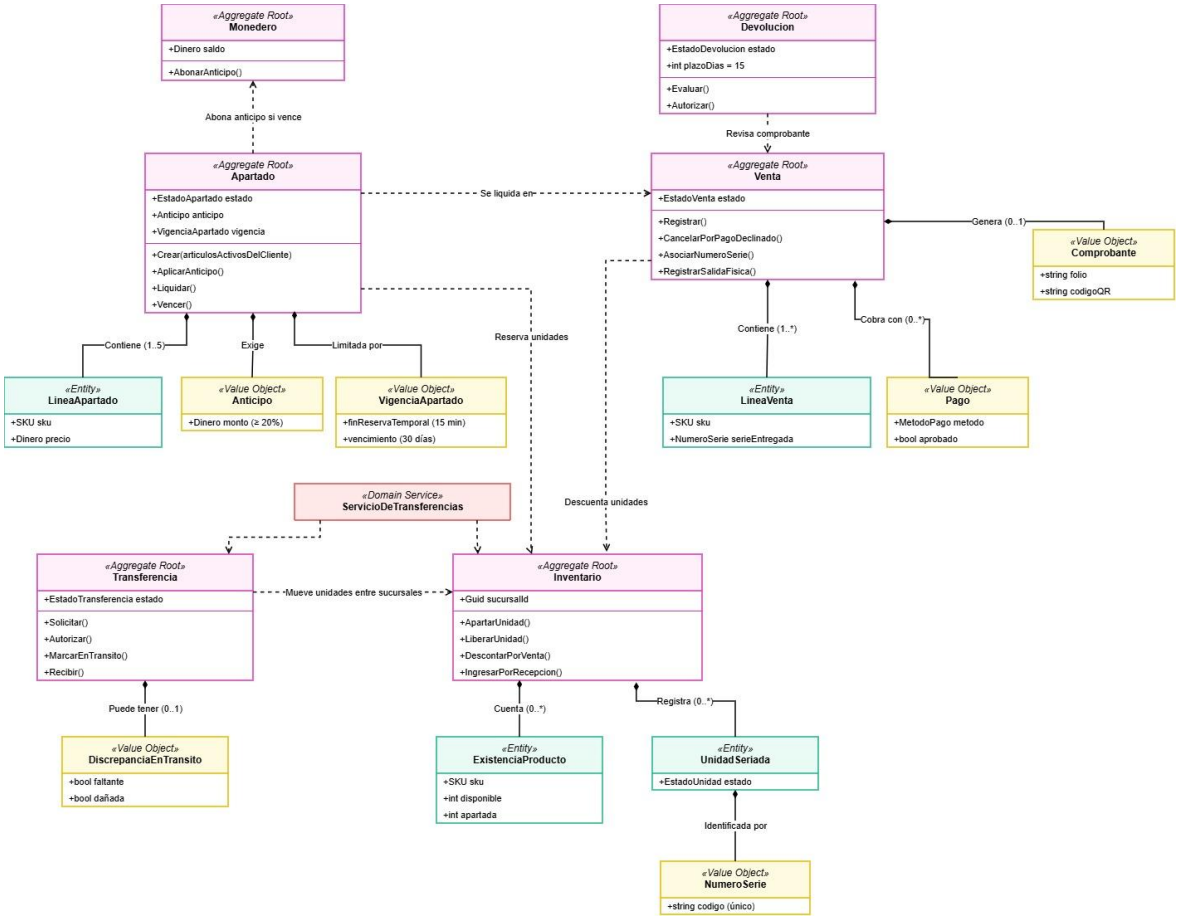


Figura 2. Modelo táctico de la U3 (seis agregados). El prototipo implementa Apartado, Inventario y Monedero.

2.2 Specification y State Machine

El patrón Specification convierte una condición del negocio en un objeto que se puede combinar con AND, OR y NOT. Así evitamos que las reglas queden regadas en `if` por todo el código, y cualquier especificación explica por qué falló. Tenemos ocho reglas simples y las clases `AndSpecification`, `OrSpecification` y `NotSpecification` para combinarlas.

Specification	Qué comprueba
`ActorActivo`	El cliente está activo.
`ActorPreferente`	El cliente es preferente.
`OperacionConLineas`	La operación tiene al menos una partida.
`OperacionAutorizada`	La operación viene autorizada.
`PromocionActiva`	Hay una promoción vigente.
`TotalMinimo`	El total llega al mínimo (1,500 MXN por omisión).
`MaximoArticulosPorCliente`	Los artículos activos más los nuevos no pasan de 5.
`AnticipoMinimo`	El pago cubre al menos el 20 % del total.

Con ellas armamos las dos reglas combinadas que pide el paquete del PIA: `puede_procesarse = ActorActivo AND OperacionConLineas AND OperacionAutorizada` y `beneficio_referencia = ActorPreferente AND TotalMinimo AND NOT PromocionActiva`. Ambas se pueden consultar desde la API (`GET /api/apartados/{id}/evaluacion`) y la interfaz las muestra con una palomita o una cruz por condición.

La máquina de estados controla el ciclo de vida del apartado. En lugar de dejar que cualquier código cambie el campo `estado`, el dominio consulta una tabla de transiciones válidas y, si la acción no está en la tabla, lanza `TransicionInvalidaError`, que la API traduce a HTTP 409.

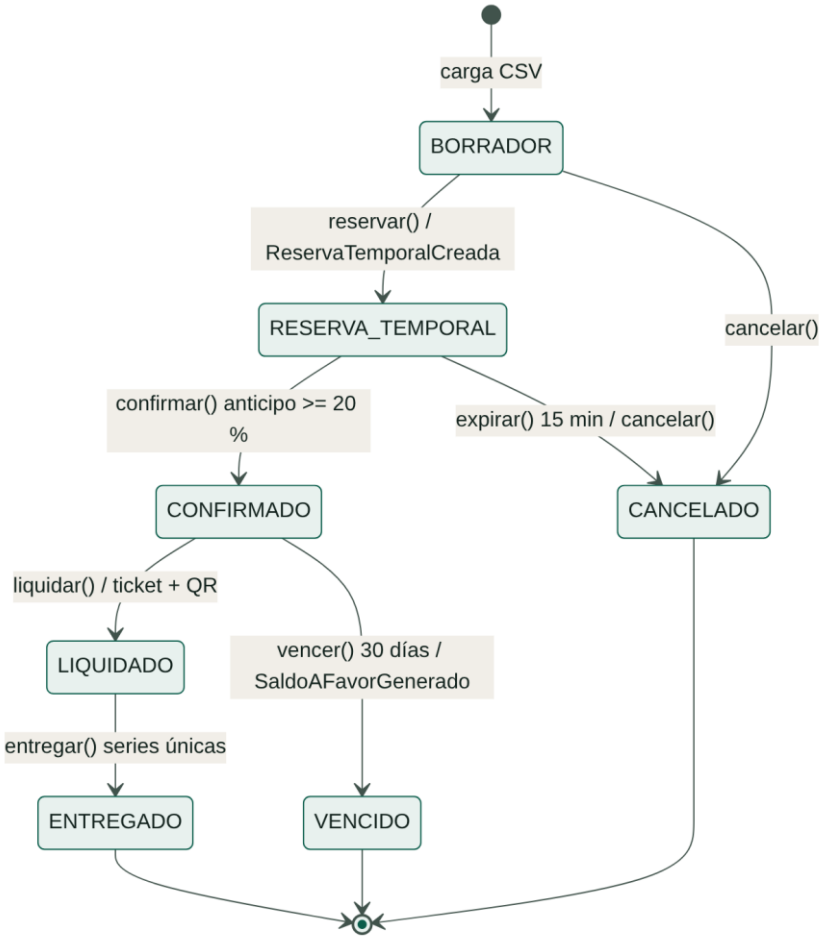


Figura 3. Máquina de estados del apartado, tal como está en `MAQUINA_APARTADO` (src/domain/estados.py).

Estado	Acciones válidas	Protección
BORRADOR	reservar, cancelar	Es el estado en que llegan las operaciones de los CSV.
RESERVA_TEMPORAL	confirmar, expirar, cancelar	No se confirma si pasaron más de 15 minutos; el pago tardío cancela y libera el stock.
CONFIRMADO	liquidar, vencer	Ya no se cancela; si pasan 30 días vence. No se liquida un vencido.
LIQUIDADO	entregar	La entrega exige una serie única por unidad.
ENTREGADO, VENCIDO, CANCELADO	ninguna	Estados finales.

Cómo mapeamos el modelo de pruebas del paquete

El paquete de datos del PIA propone un flujo de ejemplo (BORRADOR, APROBADA, SURTIDA, ENTREGADA, CANCELADA) y avisa que no es un requerimiento original. Siguiendo esa advertencia, no sustituimos nuestra máquina de estados: la **mapeamos**. APROBADA equivale a pasar por RESERVA_TEMPORAL y CONFIRMADO, SURTIDA a LIQUIDADO, ENTREGADA a ENTREGADO, y CANCELADA a CANCELADO o VENCIDO. Las diferencias completas están en `docs/architecture/diferencias.md`.

2.3 Servicios, eventos, Unit of Work y SOLID

El `ApartadoService` coordina cada caso de uso: abre una transacción con el Unit of Work, pide al dominio que aplique la regla, guarda y, ya con el commit hecho, publica los eventos. Las lecturas pasan por un `ConsultasService` aparte (CQRS básico), que no toca los agregados de escritura. Los efectos secundarios no se escriben dentro del servicio sino en handlers suscritos al bus.

Evento de dominio	Cuándo ocurre	Quién reacciona
ReservaTemporalCreada	Al reservar un borrador.	bitácora
ReservaTemporalExpirada	A los 15 minutos sin anticipo.	bitácora, liberar_stock
ApartadoConfirmado	Al pagar el anticipo.	bitácora
ApartadoLiquidado	Al pagar el saldo; trae folio y código QR.	bitácora
SalidaFisicaRegistrada	Al entregar con series.	bitácora
ApartadoCancelado	Cancelación manual.	bitácora, liberar_stock
ApartadoVencido	A los 30 días sin liquidar.	bitácora, liberar_stock, abonar_monedero
SaldoAFavorGenerado	Al abonar el anticipo al monedero.	bitácora

Principios SOLID

Principio	Cómo se aplicó
S: responsabilidad única	La API recibe HTTP, la aplicación coordina, el dominio decide y la infraestructura guarda. Cada handler hace una sola cosa.
O: abierto/cerrado	Una regla nueva es una Specification nueva que se combina con las existentes, sin editar las anteriores.
L: sustitución de Liskov	Toda Specification se usa igual desde `is_satisfied_by`, sin importar si es simple o compuesta.
I: segregación de interfaces	Cada repositorio expone solo lo que su agregado necesita (apartados, inventarios, monederos, eventos).
D: inversión de dependencias	El servicio recibe una fábrica de Unit of Work y un bus; no conoce SQL ni el motor de base de datos.

2.4 Arquitectura por capas y ADRs

El proyecto se organiza en cuatro capas con una regla simple: las dependencias apuntan hacia el dominio y el dominio no importa nada de afuera. Eso nos permitió probar casi todas las reglas sin levantar una base de datos.

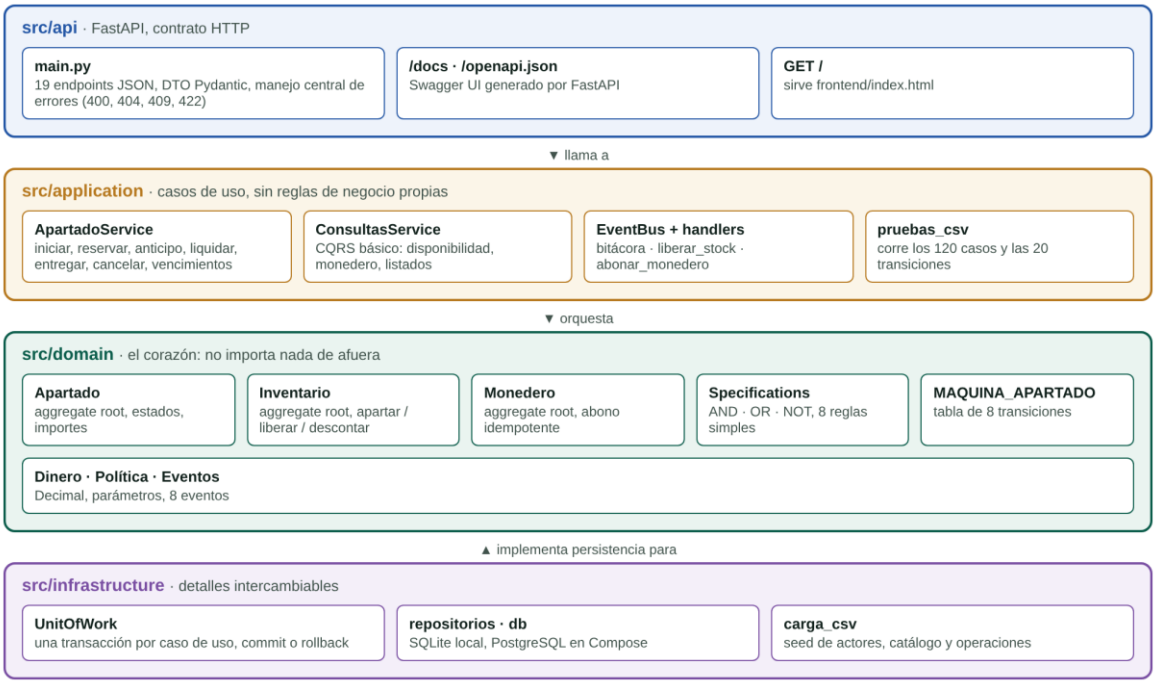


Figura 4. Arquitectura por capas del prototipo, con las piezas reales de cada carpeta de `src/`.

Las decisiones que más pesaron las dejamos en `docs/architecture/adrs.md`, con su contexto, alternativas y consecuencias:

ADR	Decisión	Consecuencia
ADR-01	Domain Model en lugar de Transaction Script o Table Module.	Más diseño al inicio; las reglas se prueban sin base de datos.
ADR-02	Unit of Work con commit y rollback, y reacciones a eventos en transacciones propias.	Apartar modifica Apartado e Inventario y no puede quedar a medias.
ADR-03	CQRS básico: las consultas pasan por `ConsultasService`.	Lectura y escritura separadas, sin bases distintas.
ADR-04	El tope de 5 artículos lo decide el dominio con un conteo que entrega el servicio.	La regla vive en el agregado, no en el endpoint.
ADR-05	Specification más tabla explícita de transiciones.	Cualquier transición fuera de la tabla termina en 409.
ADR-06	SQL portable entre SQLite y PostgreSQL; agregados serializados como JSON.	Un lock por proceso serializa escrituras; en producción con réplicas haría falta `SELECT ... FOR UPDATE`.

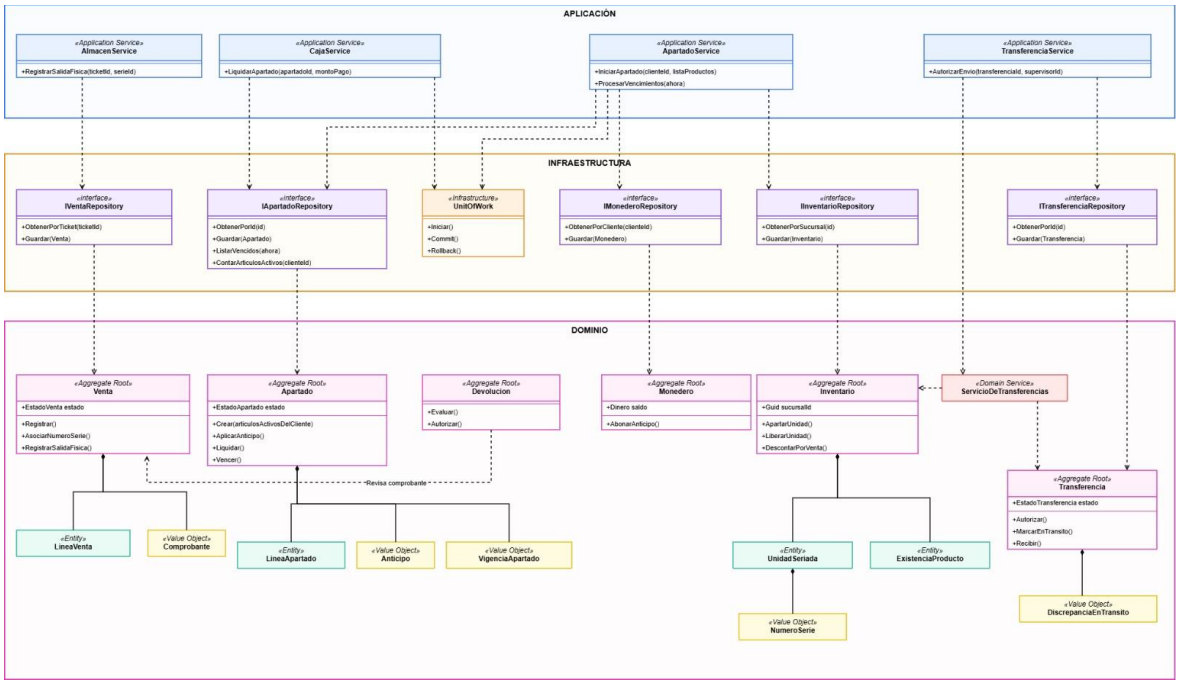


Figura 5. Diagrama de clases de la U3 por capas (diseño completo, con los seis agregados).

3. Fase de implementación

3.1 Estructura del repositorio

El repositorio sigue la estructura que pide la guía del PIA: `docs/`, `src/` dividido en dominio, aplicación, infraestructura y API, `tests/`, el workflow de CI, `docker-compose.yml`, `.env.example` y `README.md`. Los seis CSV del paquete están en `data/` y la corrida de evidencia deja sus resultados en `resultados/`.



Figura 6. Estructura del repositorio TecnoStock.

Componente	Tecnología
Lenguaje	Python 3.12 (la imagen Docker usa `python:3.12-slim`).
API	FastAPI 0.115 con documentación OpenAPI y Swagger UI automática.
Validación	Pydantic 2 en los cuerpos de las peticiones.
Persistencia	SQLite en local y PostgreSQL 15 con Docker Compose (`psycopg2`).
Pruebas	pytest, pytest-cov y el TestClient de FastAPI (httpx).
Contenedores	Docker y Docker Compose (servicios `db`, `app` y `tests`).
Integración continua	GitHub Actions (`.github/workflows/ci.yml`).
Interfaz	Un solo archivo HTML con CSS y JavaScript, sin dependencias externas.

3.2 Dominio y servicios de aplicación

Un detalle que nos costó una tarde: para poder probar los plazos de 15 minutos y 30 días sin esperar, el reloj es un parámetro. Todos los casos de uso reciben un `ahora` opcional, y la API acepta ese campo en el cuerpo de las peticiones. Con eso las pruebas viajan treinta días en

milisegundos, y la interfaz tiene un reloj simulado para la demostración. El servidor trabaja siempre en UTC.

El flujo de crear un apartado resume bien cómo colaboran las capas. La API solo traduce HTTP, el servicio abre la transacción, el dominio aplica las Specifications y la máquina de estados, el inventario aparta las unidades y, si todo salió bien, se hace commit y se publica el evento. Si algo falla en cualquier punto, el rollback deja inventario y apartado como estaban.

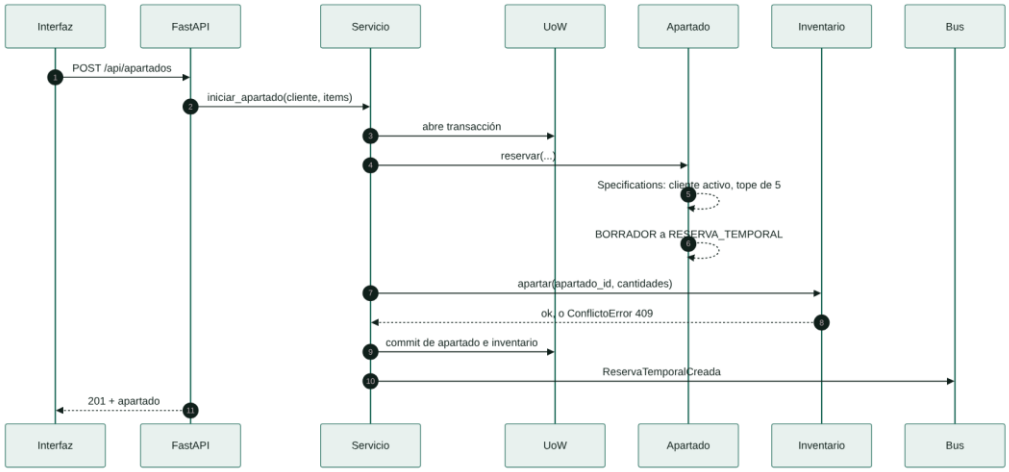


Figura 7. Secuencia de ` POST /api/apartados` : del endpoint a la transacción y al evento.

El vencimiento es el mejor ejemplo de por qué usamos eventos. El servicio solo pide vencer los apartados confirmados cuyo plazo terminó; todo lo demás (liberar el stock, abonar el monedero, anotar en la bitácora) lo hacen los handlers, cada uno en su propia transacción. El abono al monedero es idempotente: si el evento llega dos veces, el saldo no se duplica.

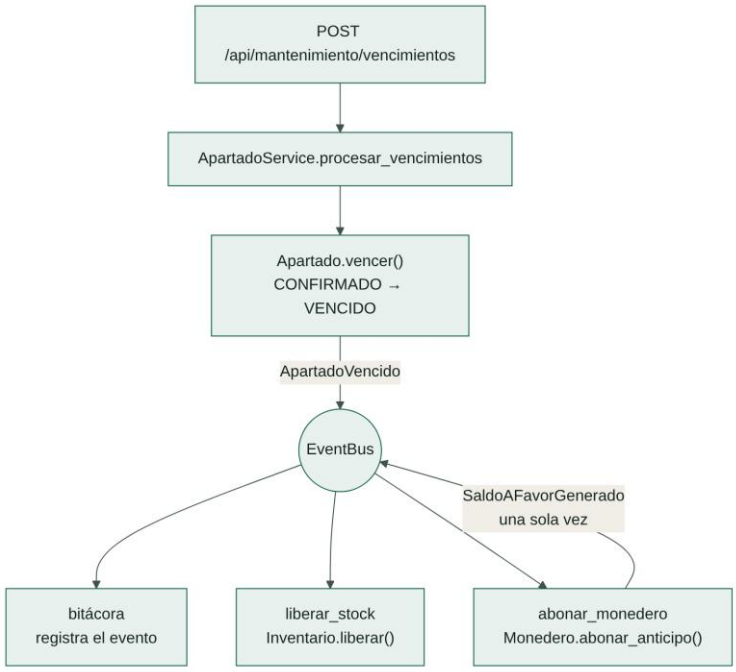


Figura 8. Reacciones al evento `ApartadoVencido` en el bus de eventos.

3.3 Persistencia y API REST

Los agregados se guardan serializados como JSON en columnas de texto y las escrituras usan SQL portable (`ON CONFLICT`), así el mismo código corre en SQLite y en PostgreSQL. Al arrancar, si `SEED_DATA` es verdadero, se cargan los CSV del paquete: 40 actores, 20 productos y 60 operaciones en borrador. La configuración llega por variables de entorno y el repositorio solo trae `.env.example`, sin secretos.

Variable	Valor por omisión	Para qué sirve
`DATABASE_URL`	SQLite local, o PostgreSQL en Compose	Conexión a la base de datos.
`SEED_DATA`	true	Carga los CSV al arrancar.
`PORCENTAJE_ANTICIPO`	0.20	Anticipo mínimo.
`LIMITE_ARTICULOS`	5	Tope de artículos activos por cliente.
`MINUTOS_RESERVA`	15	Duración de la reserva temporal.
`DIAS_VIGENCIA`	30	Vigencia del apartado confirmado.
`SUCURSAL_ID`	SUC-01	Sucursal que opera el prototipo.

La API expone 19 endpoints JSON. Swagger UI está en `/docs` y el contrato en `/openapi.json`; el script `scripts/export_openapi.py` lo guarda en `docs/api/openapi.json` y el CI lo publica como artefacto. Los errores de dominio se traducen a un cuerpo uniforme `{error, detail}`

con estos códigos: 200 (consulta o acción), 201 (creación), 400 y 422 (datos inválidos o regla de negocio), 404 (no existe) y 409 (conflicto o transición inválida).

Grupo	Endpoints principales
Sistema	`GET /health` · `GET /api/config` · `GET /` (interfaz web)
Consultas	`GET /api/actores` · `GET /api/productos` · `GET /api/productos/{id}/disponibilidad` · `GET /api/monederos/{cliente_id}` · `GET /api/eventos`
Apartados	`POST /api/apartados` · `GET /api/apartados` · `GET /api/apartados/{id}` · `GET /api/apartados/{id}/evaluacion`
Transiciones	`POST /api/apartados/{id}/reservar` · `/anticipo` · `/liquidar` · `/entregar` · `/cancelar`
Mantenimiento	`POST /api/mantenimiento/vencimientos` · `POST /api/mantenimiento/cargar-csv` · `POST /api/pruebas/ejecutar`

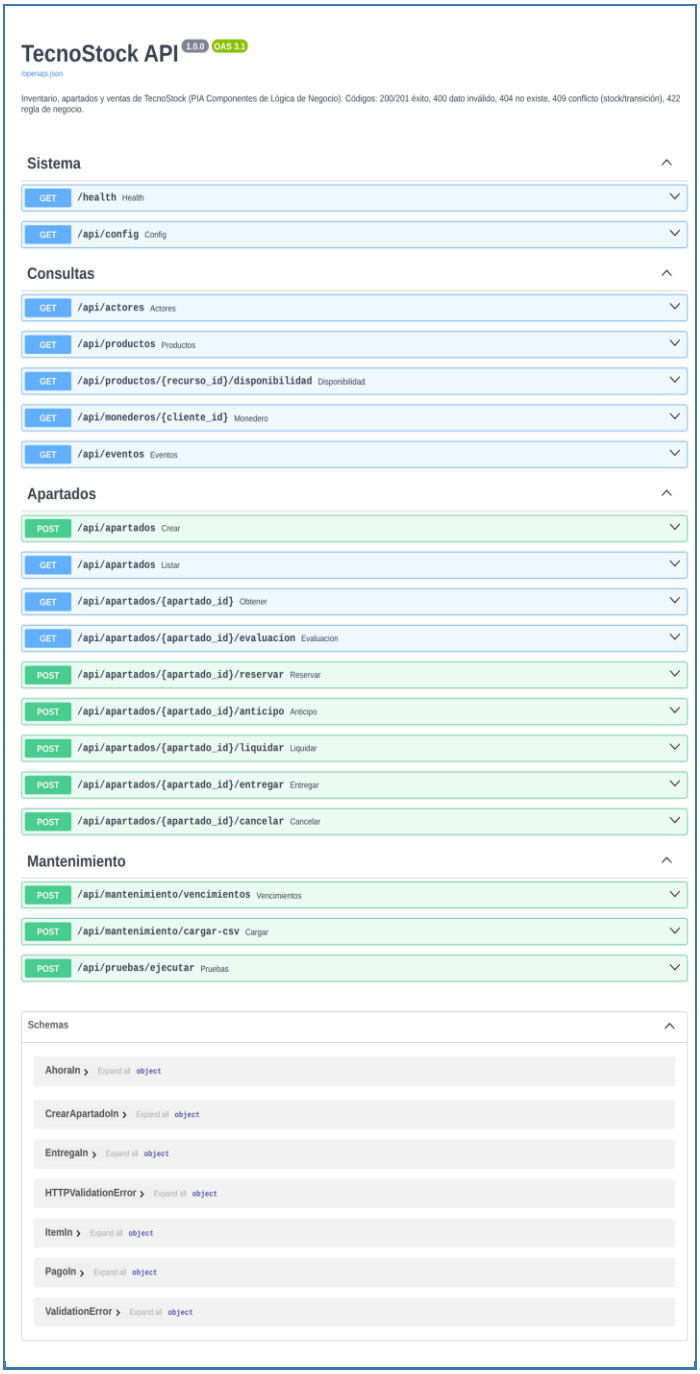


Figura 9. Swagger UI en `/docs` : todos los grupos de la API.

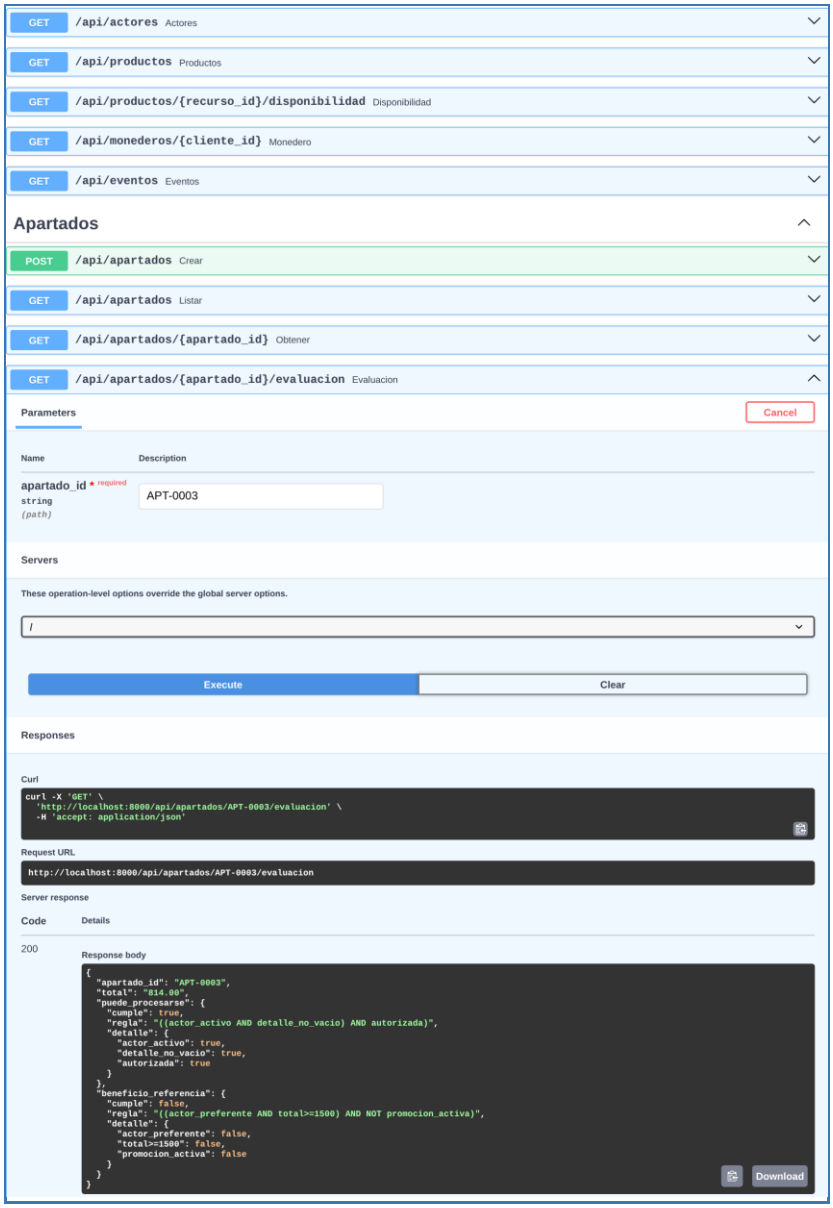


Figura 10. Ejecución real de `GET /api/apartados/{id}/evaluacion` desde Swagger: las Specifications explican cada condición.

3.4 Interfaz web

La interfaz es un solo archivo, `frontend/index.html`, que el mismo servidor FastAPI entrega en la ruta raíz. Está pensada para el mostrador y consume únicamente la API que ya existe. La primera pantalla es un tablero con una columna por estado, y de ahí se abre el detalle de cada apartado en un panel lateral desde el que se ejecutan las transiciones. Sin conexión con el servidor, la pantalla explica qué comando correr y se reconecta sola.

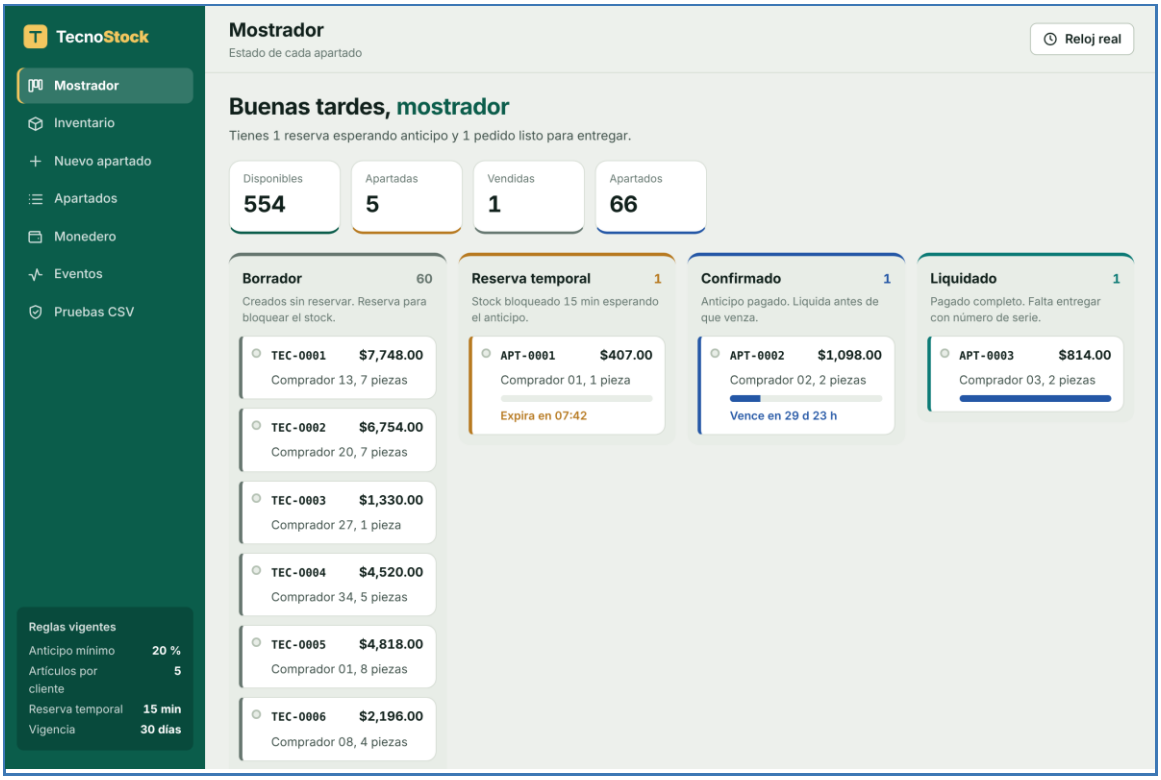


Figura 11. Mostrador: tablero por estado, con plazo restante en las reservas y confirmados.

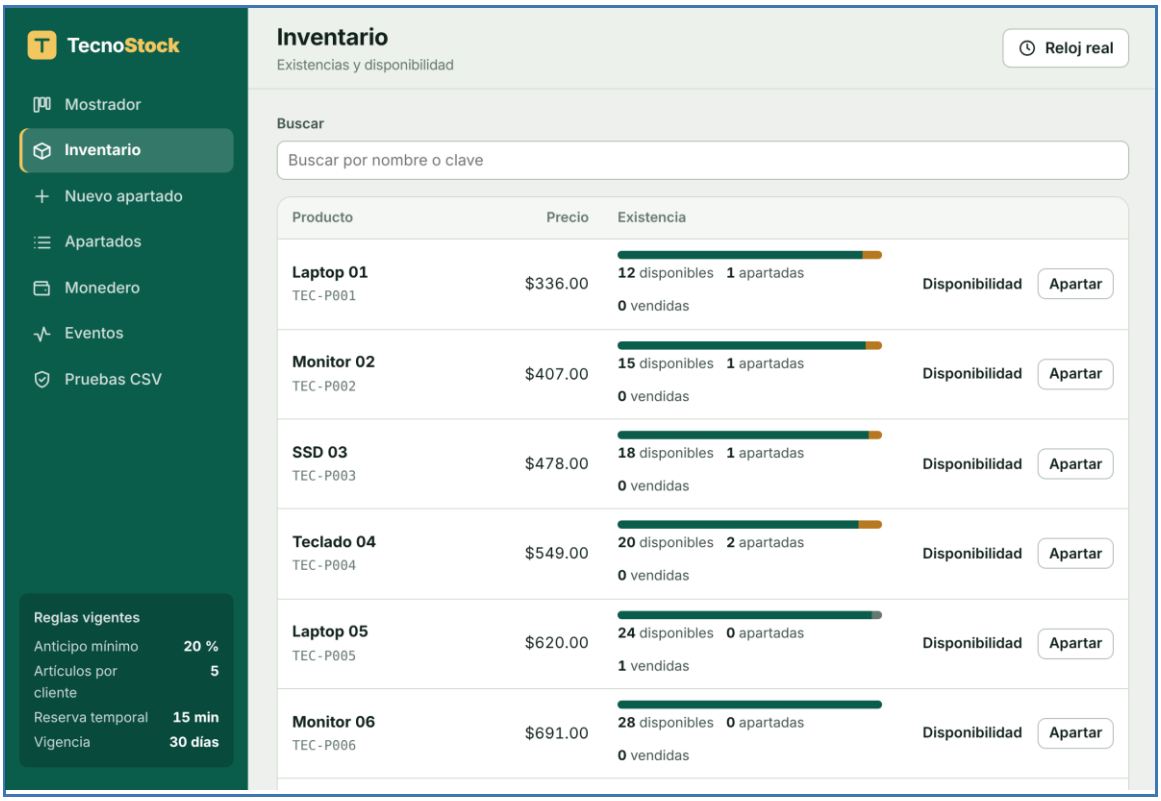


Figura 12. Inventario con existencia disponible, apartada y vendida por producto.

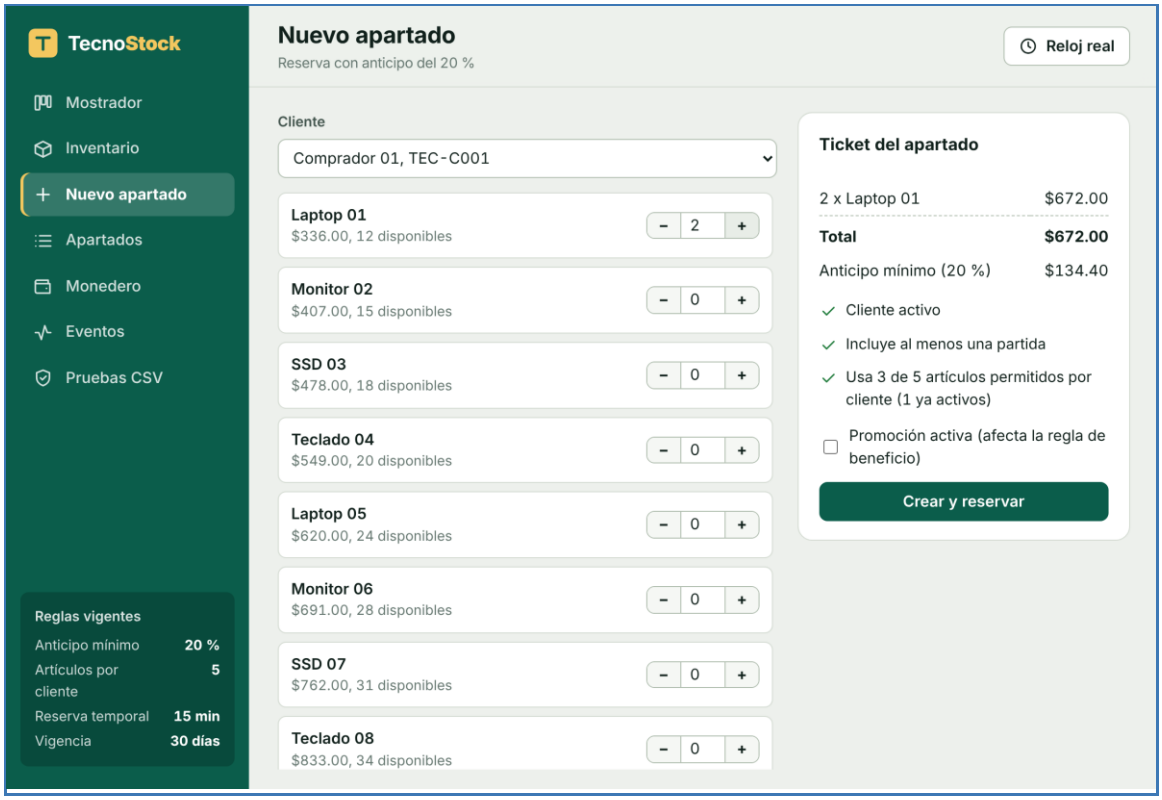


Figura 13. Nuevo apartado: el ticket calcula el anticipo y marca en vivo qué reglas se cumplen.

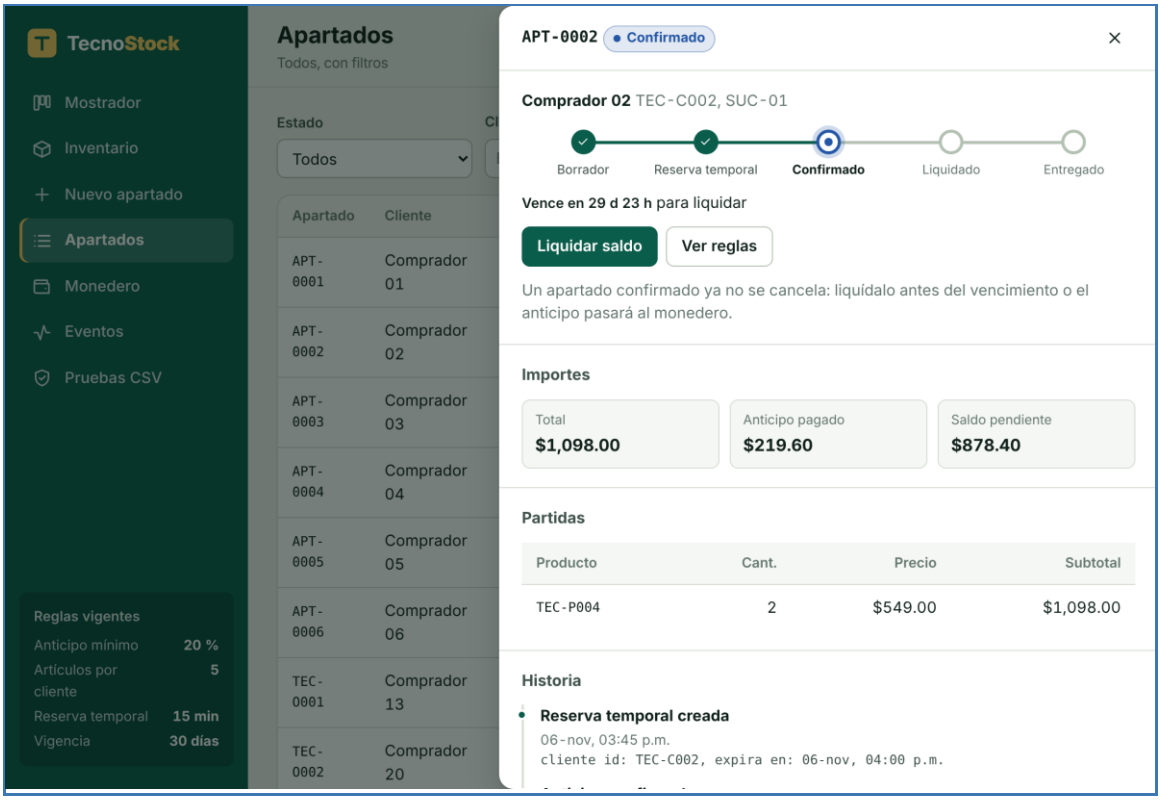


Figura 14. Detalle de un apartado confirmado: progreso por estado, importes y acciones válidas.

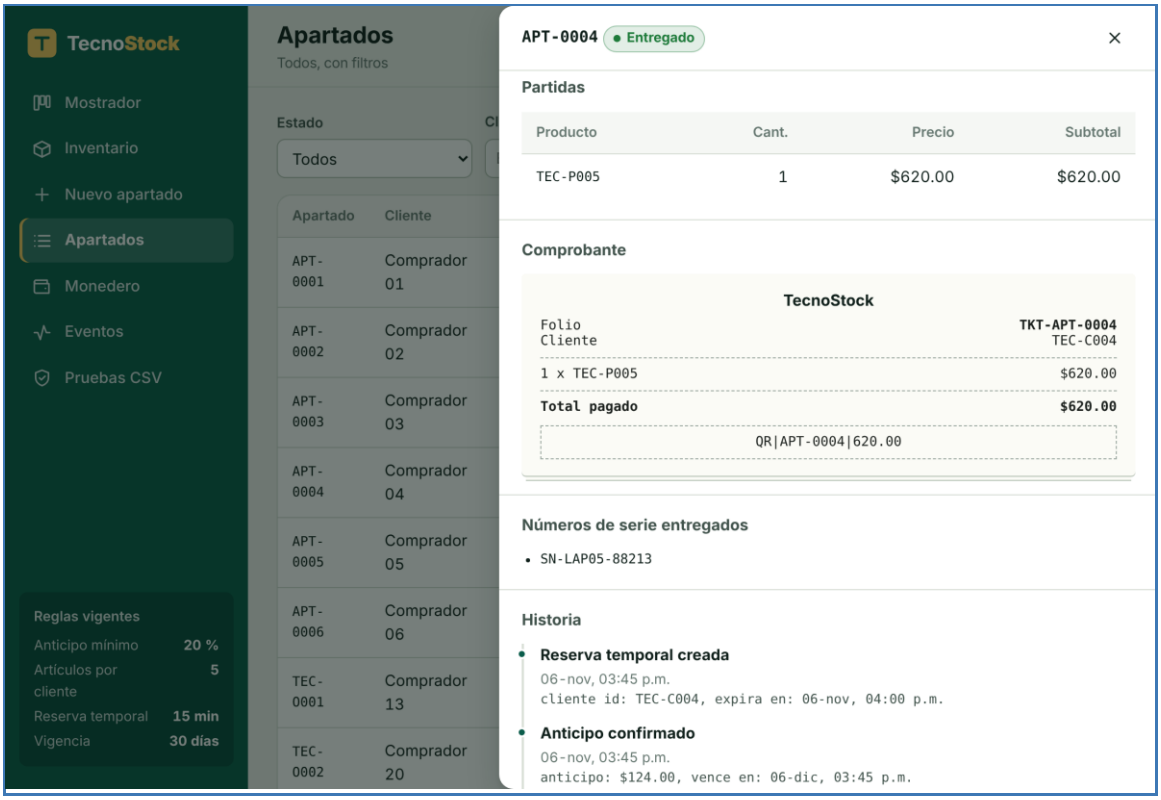


Figura 15. Apartado entregado: ticket con folio y QR, número de serie e historia de eventos.

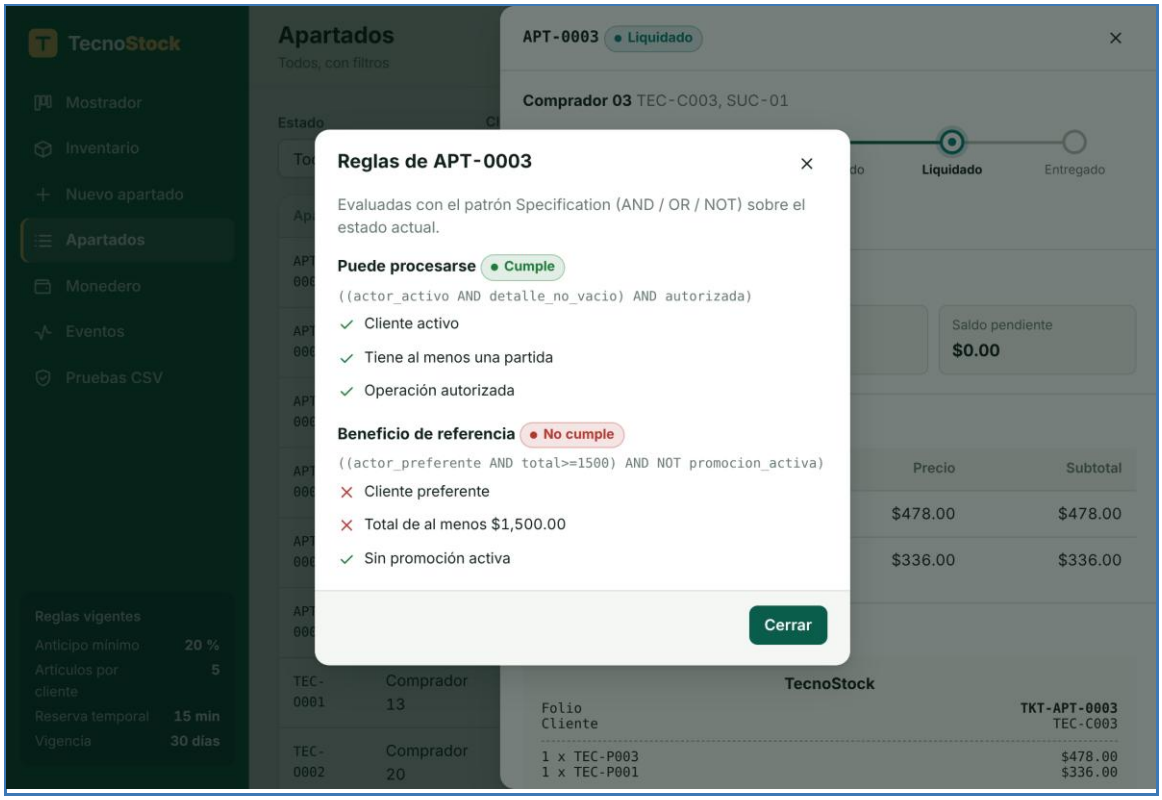


Figura 16. Las Specifications `puede_procesarse` y `beneficio_referencia` evaluadas para un apartado.

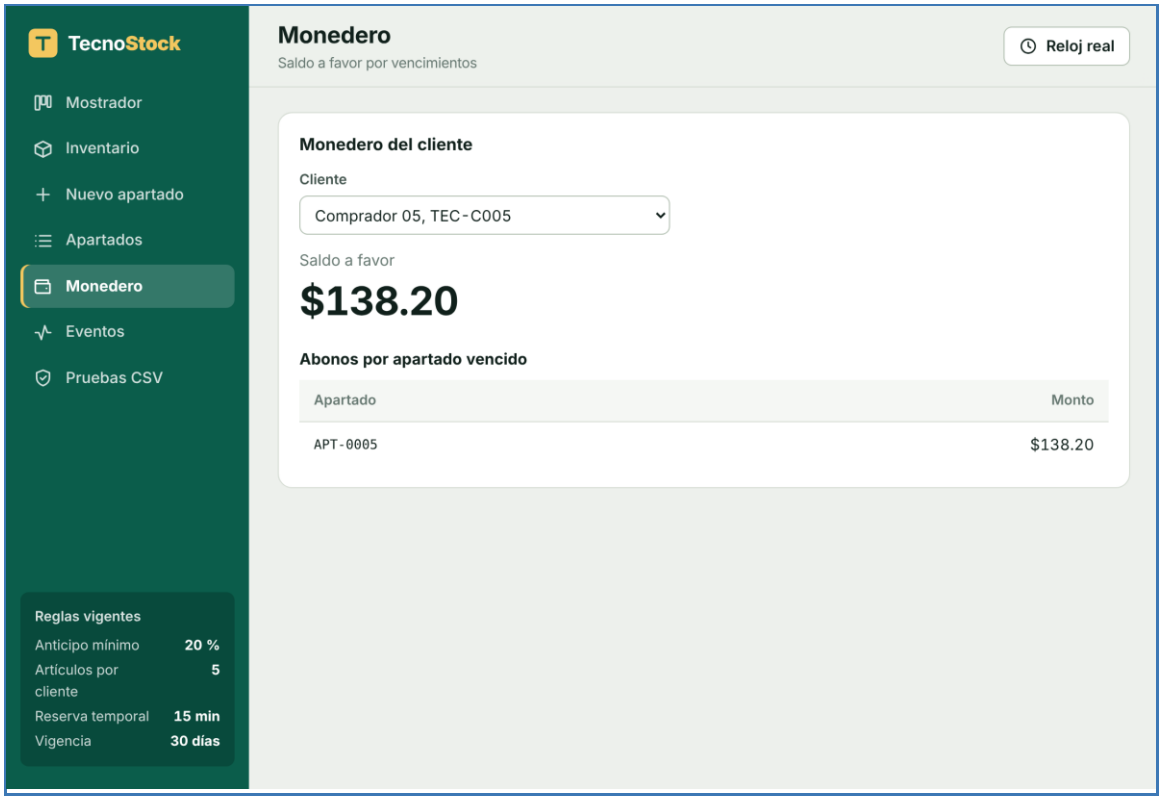


Figura 17. Monedero de un cliente con el abono de un apartado vencido.

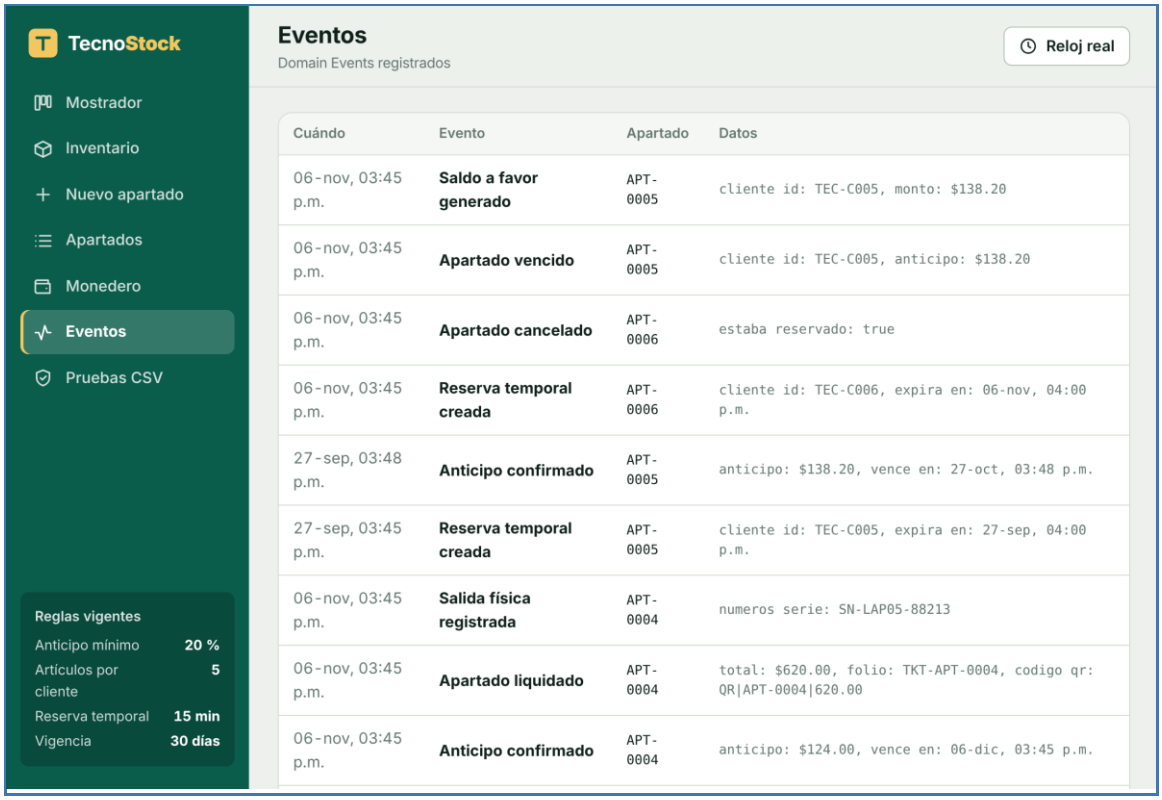


Figura 18. Bitácora de eventos de dominio.

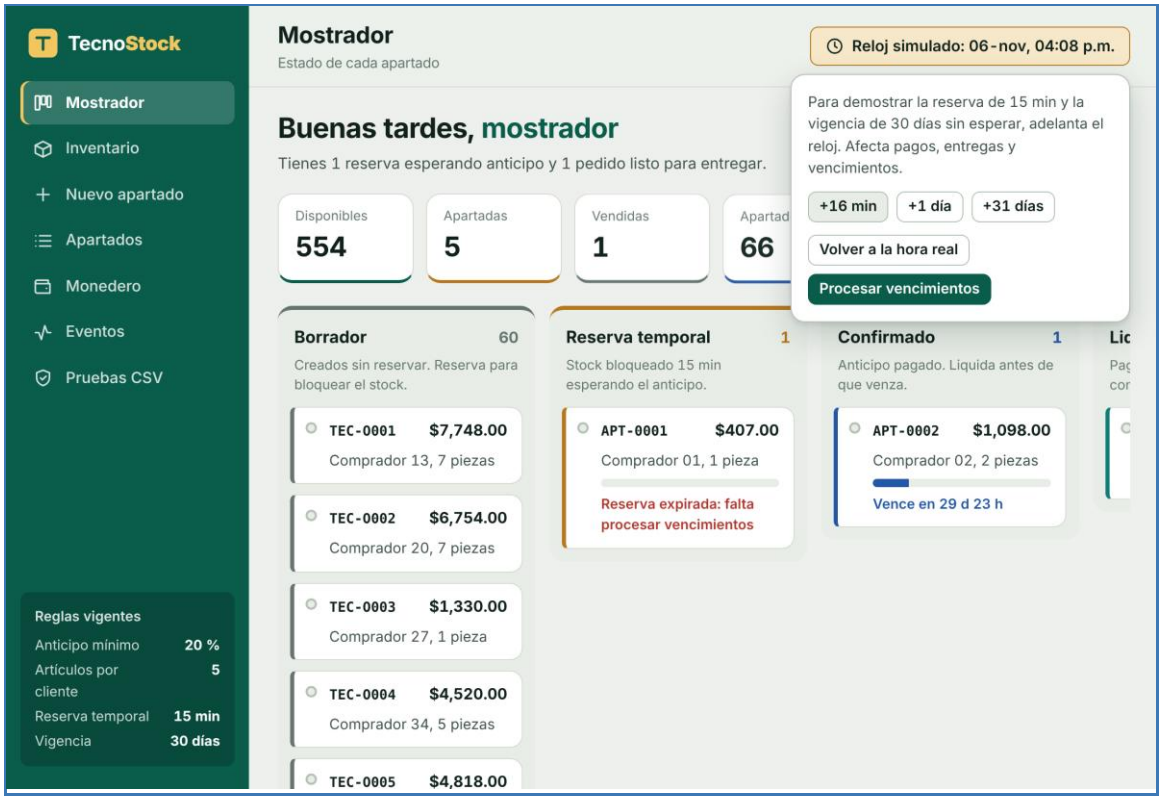


Figura 19. Reloj simulado: adelanta el tiempo para demostrar la reserva de 15 min y el vencimiento de 30 días.

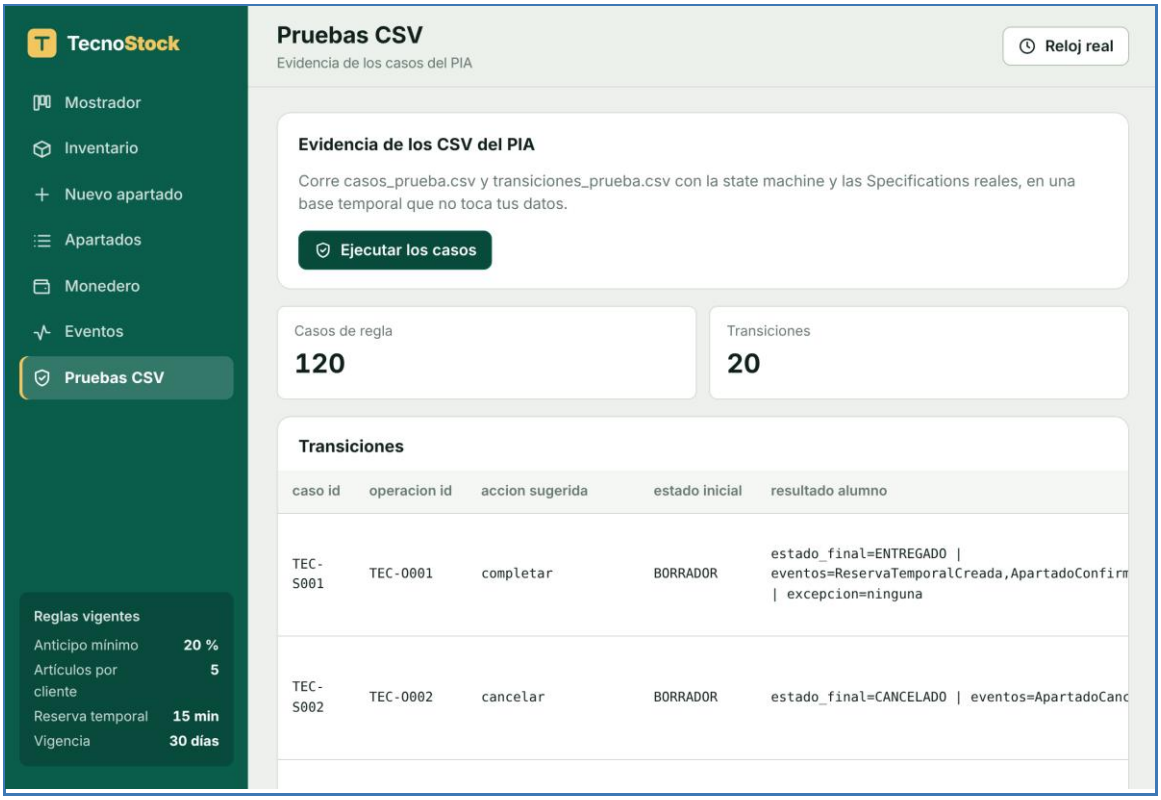


Figura 20. Pruebas CSV: ejecución de los 120 casos y las 20 transiciones del paquete del PIA.

3.5 Apoyo de inteligencia artificial

Usamos un asistente de inteligencia artificial como apoyo para revisar el código, mejorar la interfaz web y ordenar este reporte. Las decisiones de diseño (agregados, reglas, estados y plazos) salen de nuestro análisis de las unidades 2 y 3, y cada cambio lo comprobamos ejecutando la suite de pruebas y recorriendo los flujos en la interfaz. Por ejemplo, al revisar el proyecto apareció una prueba que ya fallaba porque asumía un solo apartado por cliente cuando los CSV traen operaciones en borrador, y la corregimos. Nos hacemos responsables de lo que entregamos y de poder explicarlo en la defensa.

4. Fase de calidad, integración y producción

4.1 Pruebas automatizadas y cobertura

La suite mezcla pruebas del dominio (Specifications, máquina de estados, inventario y monedero), pruebas de la capa de aplicación con base de datos real, pruebas de la API con el TestClient y pruebas del bus de eventos y la configuración. El archivo `pytest.ini` ya incluye la cobertura y falla la corrida si baja de 80 %, así que el umbral no depende de que alguien se acuerde de revisarlo.

Indicador	**Resultado validado**
Pruebas automatizadas	41 aprobadas, 0 fallidas
Cobertura de `src/`	98.37 %
Cobertura mínima solicitada	80 %
Resultado	Cumple el umbral de cobertura

Archivo de pruebas	**Pruebas**	**Qué cubre**
`test_dominio.py`	18	Specifications AND/OR/NOT, máquina de estados, reglas del apartado, series, inventario y monedero idempotente.
`test_aplicacion.py`	16	Casos de uso con base de datos: reserva, expiración, vencimiento, rollback, límite de 5 y corrida de los CSV.
`test_api.py`	5	Endpoints y códigos HTTP 201, 400, 404, 409 y 422; flujo completo y la interfaz en `/`.
`test_bus.py`, `test_config.py`	2	Bus de eventos y lectura de variables de entorno.

Reporte

```
pytest -v

collected 41 items

tests/test_api.py::TestAPI::test_borrador_csv_reservar_cancelar_y_pruebas PASSED [  2%]
tests/test_api.py::TestAPI::test_codigos_http PASSED                       [  4%]
tests/test_api.py::TestAPI::test_flujo_completo PASSED                     [  7%]
tests/test_api.py::TestAPI::test_health_y_consultas PASSED                 [  9%]
tests/test_api.py::TestAPI::test_vencimiento_y_monedero PASSED             [ 12%]
tests/test_aplicacion.py::TestCasosDeUso::test_anticipo_insuficiente_no_cambia_nada PASSED [ 14%]
tests/test_aplicacion.py::TestCasosDeUso::test_cancelar_libera_stock PASSED [ 17%]
tests/test_aplicacion.py::TestCasosDeUso::test_cu01_disponibilidad PASSED [ 19%]
tests/test_aplicacion.py::TestCasosDeUso::test_cu02_iniciar_apartado_bloquea_stock PASSED [ 21%]
tests/test_aplicacion.py::TestCasosDeUso::test_evaluar_y_listar PASSED     [ 24%]
tests/test_aplicacion.py::TestCasosDeUso::test_flujo_completo_y_salida_fisica PASSED [ 26%]
tests/test_aplicacion.py::TestCasosDeUso::test_id_duplicado PASSED         [ 29%]
tests/test_aplicacion.py::TestCasosDeUso::test_limite_5_global_por_cliente PASSED [ 31%]
tests/test_aplicacion.py::TestCasosDeUso::test_no_se_vende_lo_apartado PASSED [ 34%]
tests/test_aplicacion.py::TestCasosDeUso::test_pago_tardio_cancela_y_rechaza PASSED [ 36%]
tests/test_aplicacion.py::TestCasosDeUso::test_rechazos_de_creacion PASSED [ 39%]
tests/test_aplicacion.py::TestCasosDeUso::test_reservar_borrador_y_rollback_uow PASSED [ 41%]
tests/test_aplicacion.py::TestCasosDeUso::test_stock_insuficiente_409 PASSED [ 43%]
tests/test_aplicacion.py::TestCasosDeUso::test_us09_reserva_expirada_libera PASSED [ 46%]
tests/test_aplicacion.py::TestCasosDeUso::test_us12_vencimiento_monedero_idempotente PASSED [ 48%]
tests/test_aplicacion.py::TestCargaYCsv::test_runner_csv PASSED            [ 51%]
tests/test_bus.py::TestBus::test_handler_fallido_no_rompe PASSED           [ 53%]
tests/test_config.py::TestConfig::test_env PASSED                          [ 56%]
tests/test_dominio.py::TestSpecifications::test_anticipo_minimo PASSED     [ 58%]
tests/test_dominio.py::TestSpecifications::test_beneficio_and_not PASSED   [ 60%]
tests/test_dominio.py::TestSpecifications::test_limite_articulos PASSED    [ 63%]
tests/test_dominio.py::TestSpecifications::test_or_y_explicar PASSED       [ 65%]
tests/test_dominio.py::TestSpecifications::test_puede_procesarse_and PASSED [ 68%]
tests/test_dominio.py::TestMaquinaEstados::test_cancelar PASSED            [ 70%]
tests/test_dominio.py::TestMaquinaEstados::test_entrega_series PASSED      [ 73%]
tests/test_dominio.py::TestMaquinaEstados::test_expirar_y_vencer PASSED    [ 75%]
tests/test_dominio.py::TestMaquinaEstados::test_flujo_feliz PASSED         [ 78%]
tests/test_dominio.py::TestMaquinaEstados::test_lineas_validas PASSED      [ 80%]
tests/test_dominio.py::TestMaquinaEstados::test_liquidar_reglas PASSED     [ 82%]
tests/test_dominio.py::TestMaquinaEstados::test_reserva_y_anticipo_reglas PASSED [ 85%]
tests/test_dominio.py::TestMaquinaEstados::test_reservar_rechazos PASSED   [ 87%]
tests/test_dominio.py::TestMaquinaEstados::test_total_decimal_exacto PASSED [ 90%]
tests/test_dominio.py::TestMaquinaEstados::test_transiciones_invalidas PASSED [ 92%]
tests/test_dominio.py::TestInventarioMonedero::test_apartar_liberar_vender PASSED [ 95%]
tests/test_dominio.py::TestInventarioMonedero::test_errores PASSED         [ 97%]
tests/test_dominio.py::TestInventarioMonedero::test_monedero_idempotente PASSED [100%]
```

Figura 21. Salida de `pytest -v` : las 41 pruebas aprobadas.

Reporte

pytest --cov=src (pytest.ini exige >= 80 %)

```
---------- coverage: platform linux, python 3.13.18-final-0 ----------
Name                                 Stmts   Miss  Cover   Missing
------------------------------------------------------------------
src/__init__.py                          0      0   100%
src/api/__init__.py                      0      0   100%
src/api/main.py                        127      0   100%
src/application/__init__.py              0      0   100%
src/application/apartado_service.py    159      0   100%
src/application/eventos.py              20      0   100%
src/application/handlers.py             23      0   100%
src/application/pruebas_csv.py          54      0   100%
src/config.py                           18      0   100%
src/domain/__init__.py                   0      0   100%
src/domain/actor.py                     19      0   100%
src/domain/apartado.py                 139      0   100%
src/domain/dinero.py                     6      0   100%
src/domain/errores.py                    6      0   100%
src/domain/estados.py                   31      0   100%
src/domain/eventos.py                   59      0   100%
src/domain/inventario.py                46      0   100%
src/domain/monedero.py                  26      0   100%
src/domain/politicas.py                 10      0   100%
src/domain/specifications.py            71      0   100%
src/infrastructure/__init__.py           0      0   100%
src/infrastructure/carga_csv.py         32      0   100%
src/infrastructure/db.py                28      2    93%   24-25
src/infrastructure/repositorios.py     118      1    99%   84
src/infrastructure/unit_of_work.py      22      3    86%   19-21
src/tools_run_csv.py                    30     11    63%   26-34, 38-39
------------------------------------------------------------------
TOTAL                                 1044     17    98%

Required test coverage of 80% reached. Total coverage: 98.37%
======================== 41 passed, 2 warnings in 1.71s ========================
```

Figura 22. Reporte de cobertura por archivo: 98.37 % en total.

La cobertura solo dice qué porcentaje del código se ejecutó; no prueba por sí sola que los requisitos estén completos. Por eso la complementamos con la matriz de trazabilidad (sección 4.6) y con la demostración de los flujos.

Evidencia con los CSV del PIA

Con `POST /api/pruebas/ejecutar` (o la pestaña Pruebas CSV de la interfaz) corremos los 120 casos de `casos_prueba.csv` y las 20 pruebas de `transiciones_prueba.csv` en una base temporal que no toca los datos reales. El resultado de la última corrida fue el siguiente:

Conjunto	Resultado
120 casos de Specification (60 `validar_operacion` y 60 `evaluar_beneficio`)	57 CUMPLE y 63 NO_CUMPLE, cada uno con el detalle de qué condición falló.
20 transiciones (aprobar, completar, cancelar, repetir_aprobacion y saltar_estado, 4 de cada una)	8 terminan sin error. 12 se rechazan como corresponde: 6 por transición inválida, 3 por producto deshabilitado y 3 porque la operación no puede procesarse.

Algo que aprendimos aquí: las operaciones del CSV traen hasta 9 piezas, y nuestro tope de 5 artículos por cliente las habría rechazado todas. Desactivamos el tope **solo en la corrida de evidencia** (`src/application/pruebas_csv.py`) y la API lo aplica siempre. Esa diferencia, y las demás, están documentadas en `docs/architecture/diferencias.md`.

4.2 Docker Compose y ejecución local

Para correr sin Docker basta con `pip install -r requirements.txt` y `uvicorn src.api.main:app --reload`; la aplicación queda en `http://localhost:8000` y usa SQLite. Con Docker, el archivo `docker-compose.yml` define tres servicios: `db` (PostgreSQL 15 con healthcheck), `app` (la API, que espera a que la base esté saludable) y `tests`, un perfil opcional que ejecuta pytest con la cobertura mínima.

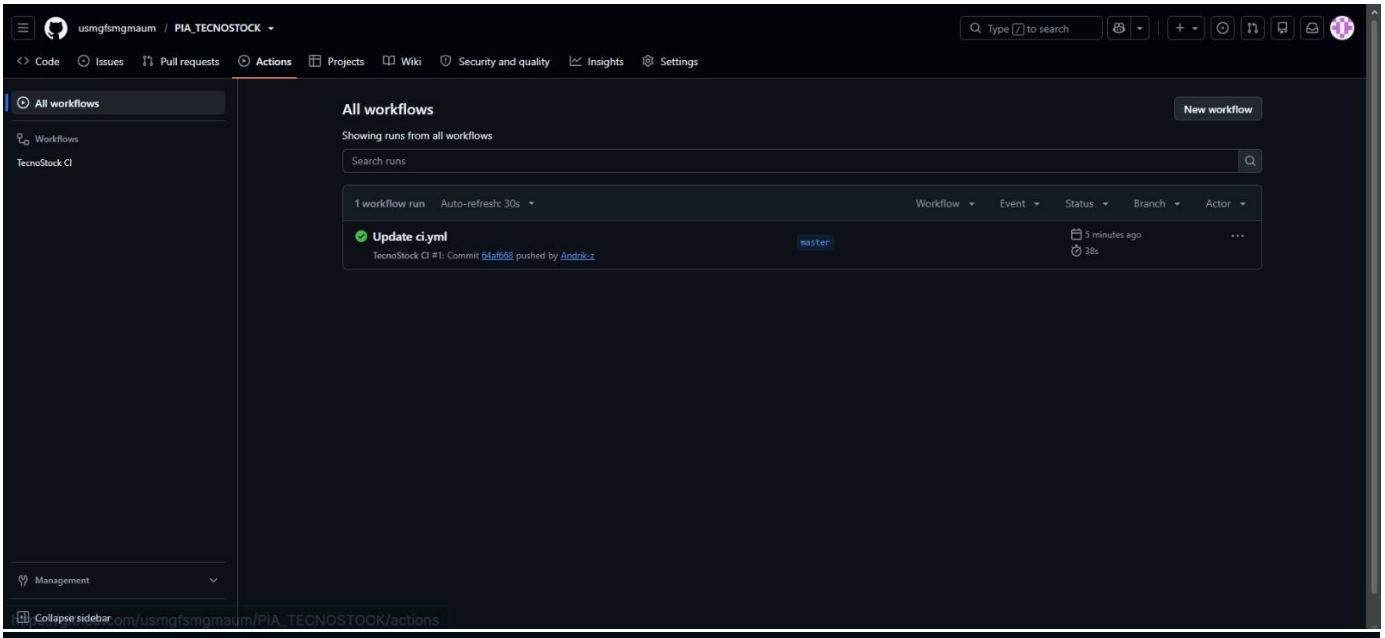
Para qué	Comando
Levantar base de datos y API	`docker compose up --build`
Ejecutar las pruebas en contenedor	`docker compose --profile tests run --rm tests`
Ejecutar sin Docker	`uvicorn src.api.main:app --reload`
Revisar salud de la API	`GET http://localhost:8000/health`

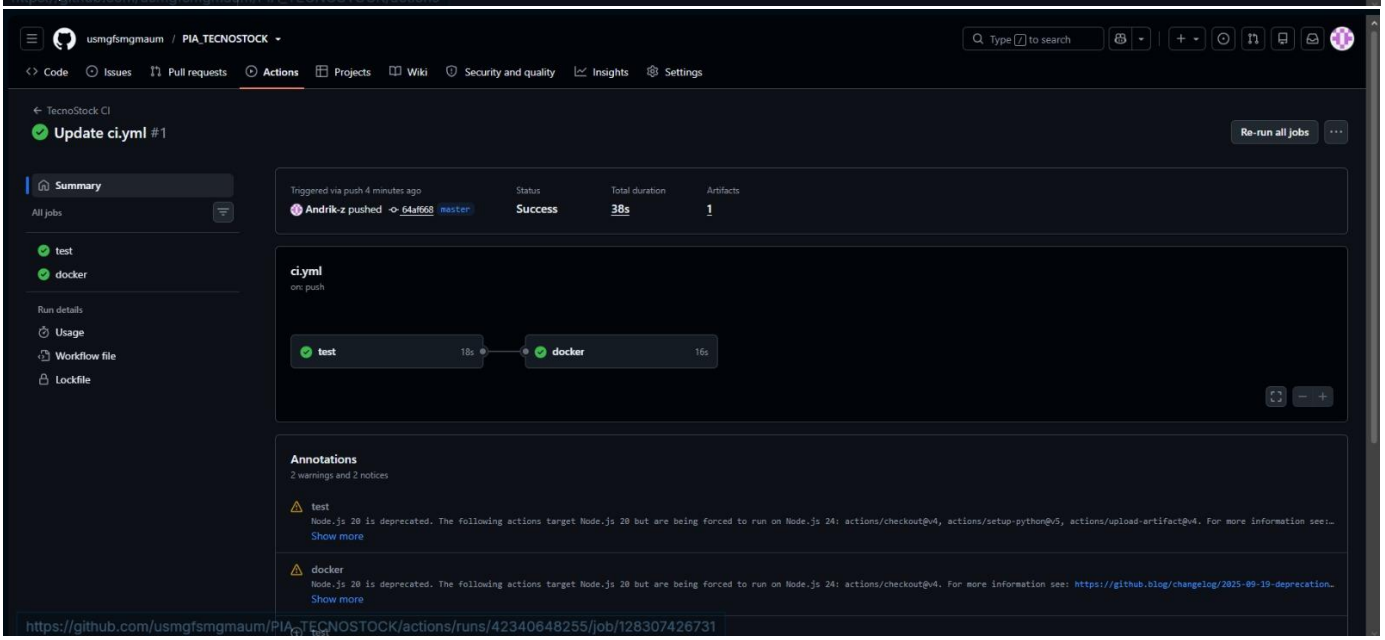
4.3 Integración continua (CI/CD)

El workflow `.github/workflows/ci.yml` se dispara con cada push y pull request hacia `main`. Tiene dos trabajos. El primero instala dependencias, corre pytest con el umbral de cobertura, exporta el contrato OpenAPI y lo guarda como artefacto. El segundo, que depende del primero, construye la imagen Docker y vuelve a correr las pruebas dentro del contenedor.

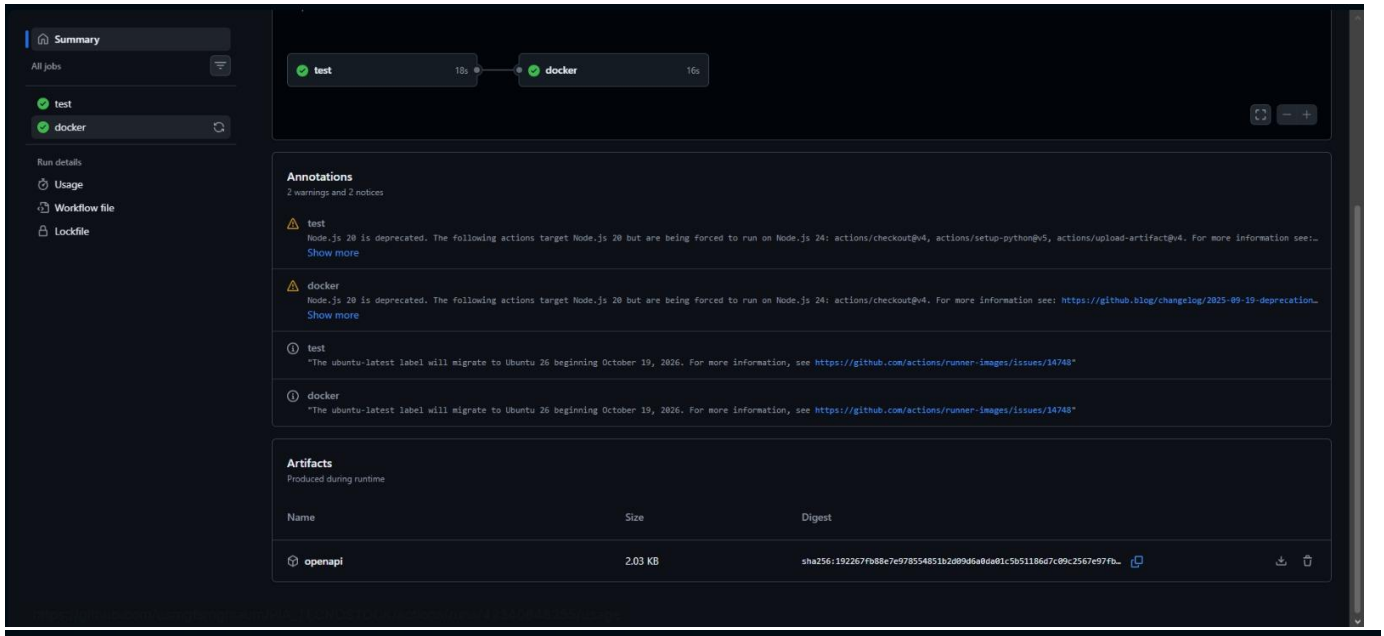
Repositorio del proyecto: https://github.com/usmgfsmgmaum/PIA_TECNOSTOCK

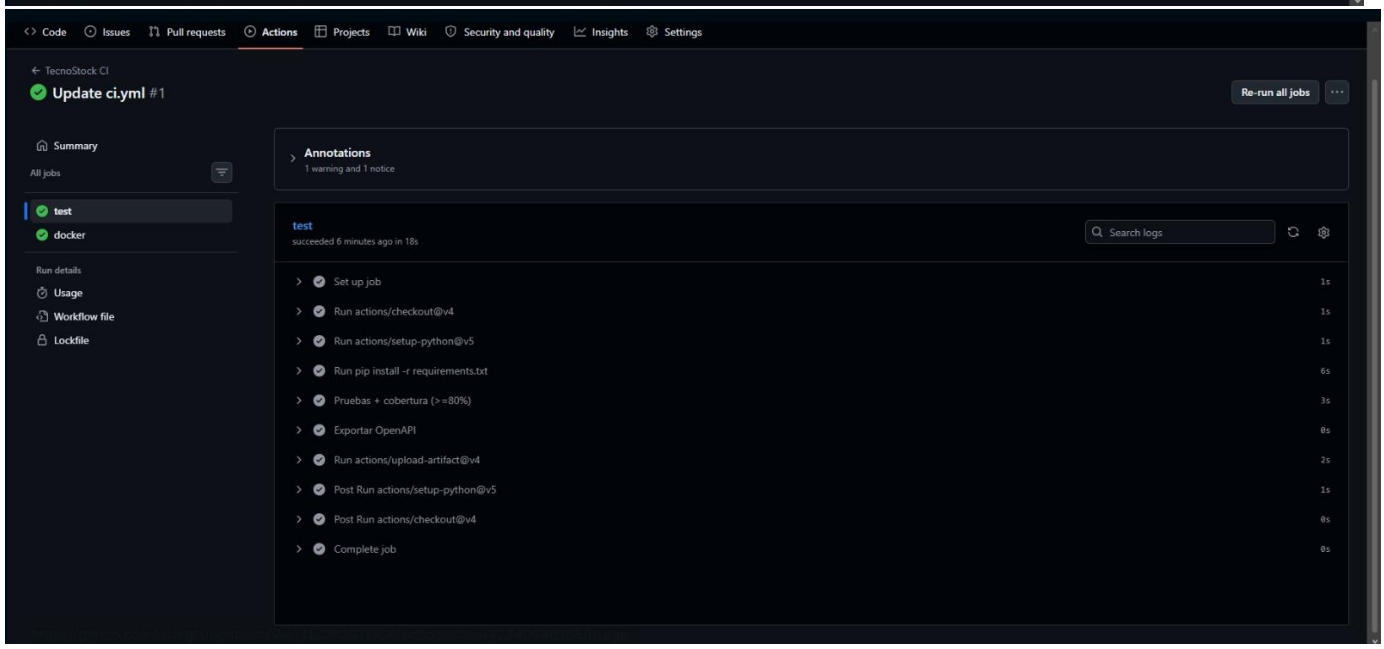
Reporte

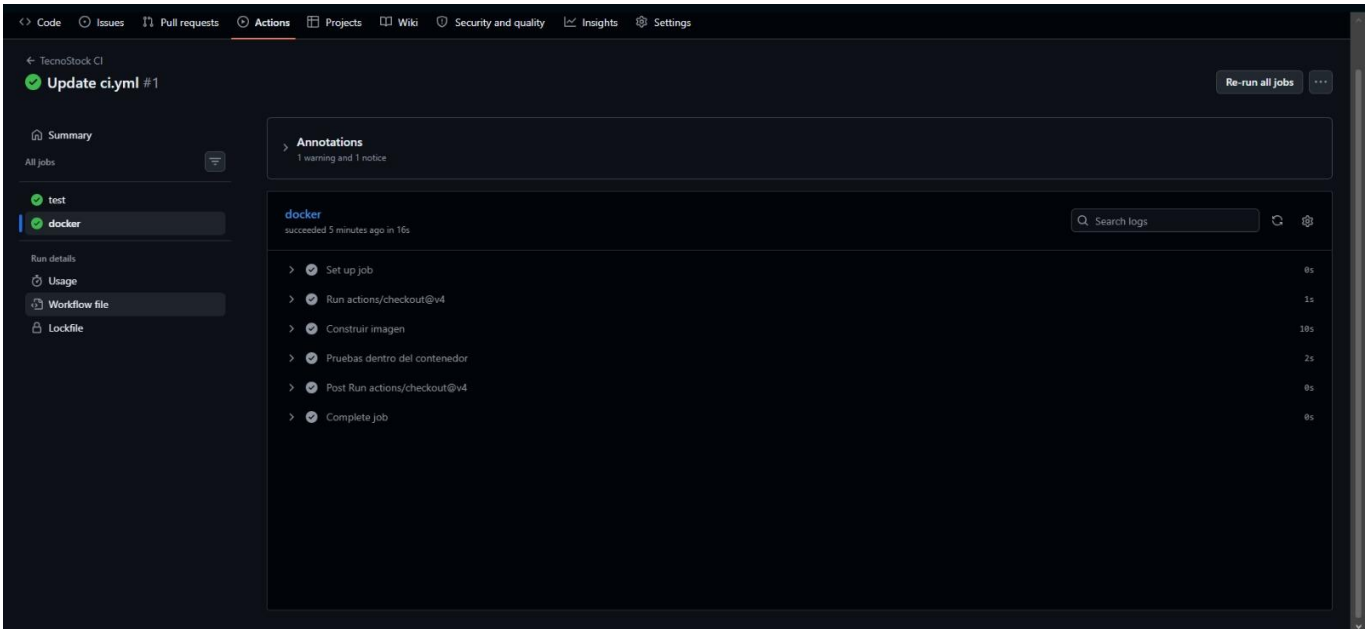

usmgfsmgmaum / PIA_TECNOSTOCK
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
All workflows
Workflows
TecnoStock CI
Management
Collapse sidebar
All workflows
New workflow
Showing runs from all workflows
Search runs
1 workflow run
Auto-refresh: 30s
Workflow
Event
Status
Branch
Actor
Update ci.yml
TecnoStock CI #1: Commit 64af668 pushed by Andrik-z
master
5 minutes ago
38s
https://github.com/usmgfsmgmaum/PIA_TECNOSTOCK/actions


usmgfsmgmaum / PIA_TECNOSTOCK
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
TecnoStock CI
Update ci.yml #1
Re-run all jobs
Summary
All jobs
test
docker
Run details
Usage
Workflow file
Lockfile
Triggered via push 4 minutes ago
Andrik-z pushed 64af668 master
Status
Success
Total duration
38s
Artifacts
1
ci.yml
on: push
test
18s
docker
16s
Annotations
2 warnings and 2 notices
test
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5, actions/upload-artifact@v4. For more information see:…
Show more
docker
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: https://github.blog/changelog/2025-09-19-deprecation…
Show more
https://github.com/usmgfsmgmaum/PIA_TECNOSTOCK/actions/runs/42340648255/job/128307426731

Reporte


Summary
All jobs
test
docker
Run details
Usage
Workflow file
Lockfile
test 18s
docker 16s
Annotations
2 warnings and 2 notices
test
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5, actions/upload-artifact@v4. For more information see:…
Show more
docker
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: https://github.blog/changelog/2025-09-19-deprecation…
Show more
test
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"
docker
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"
Artifacts
Produced during runtime
Name
Size
Digest
openapi
2.03 KB
sha256:192267fb88e7e978554851b2d09d6a0da01c5b51186d7c09c2567e97fb…


Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
TecnoStock CI
Update ci.yml #1
Re-run all jobs
Summary
All jobs
test
docker
Run details
Usage
Workflow file
Lockfile
Annotations
1 warning and 1 notice
test
succeeded 6 minutes ago in 18s
Search logs
Set up job 1s
Run actions/checkout@v4 1s
Run actions/setup-python@v5 1s
Run pip install -r requirements.txt 6s
Pruebas + cobertura (>=80%) 3s
Exportar OpenAPI 0s
Run actions/upload-artifact@v4 2s
Post Run actions/setup-python@v5 1s
Post Run actions/checkout@v4 0s
Complete job 0s



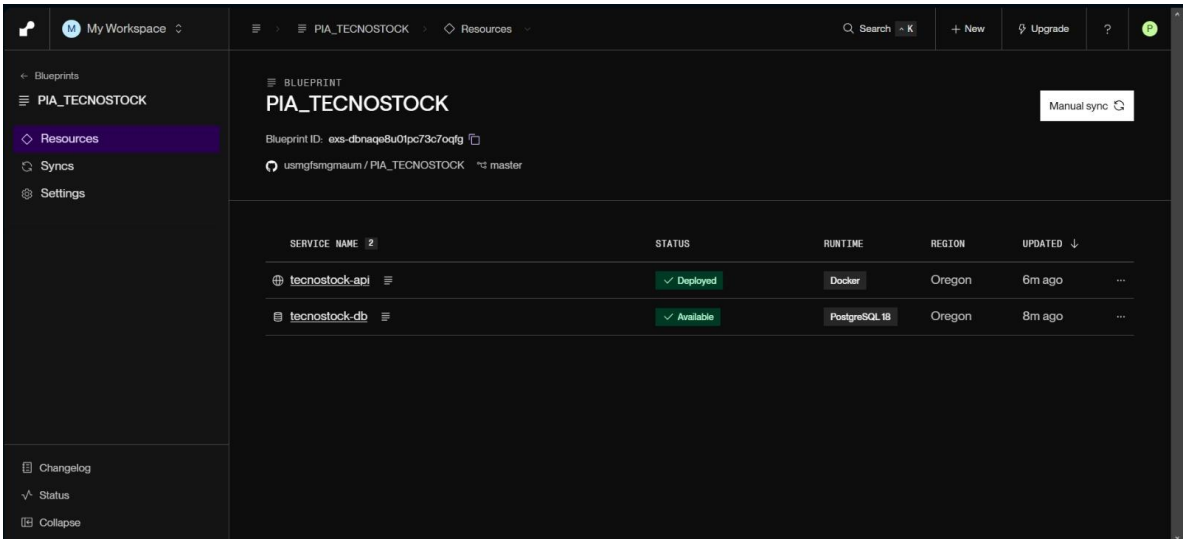
4.4 Despliegue y documentación de la API

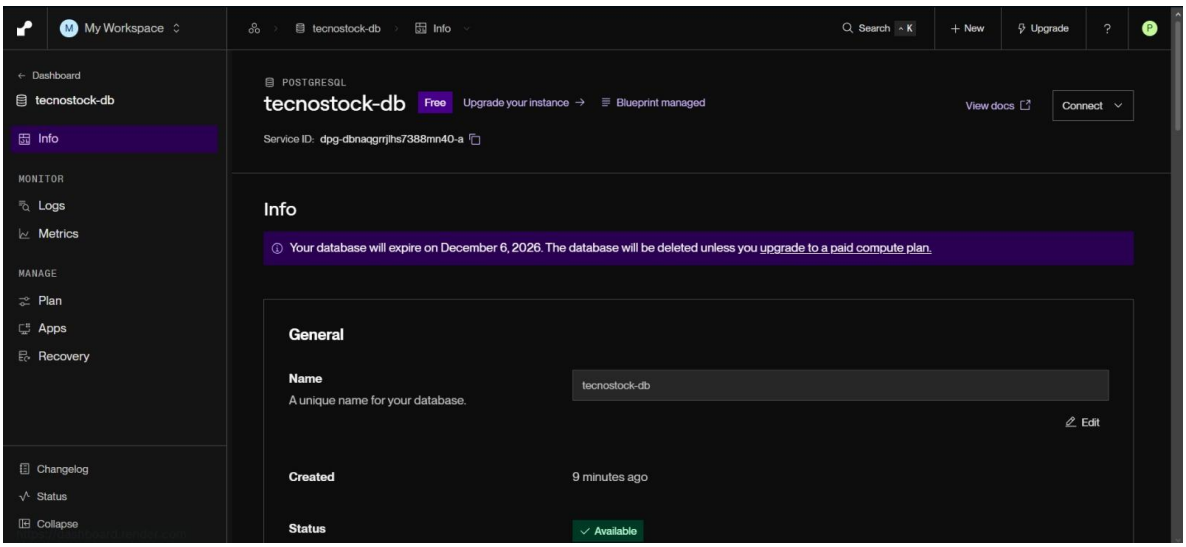
La documentación de la API se genera sola: Swagger UI en `/docs`, Redoc en `/redoc` y el contrato en `/openapi.json`, con una copia versionada en `docs/api/openapi.json`. Para la defensa, la aplicación corre en local o con Docker Compose y se demuestra desde Swagger y desde la interfaz. El repositorio de GitHub incluye una plantilla `render.yaml` para un despliegue en Render, pero **no desplegamos** el servicio, así que no hay una URL pública que citar.

El servicio desplegado en Render expone la aplicación y la documentación Swagger/OpenAPI. Los enlaces de demostración son:

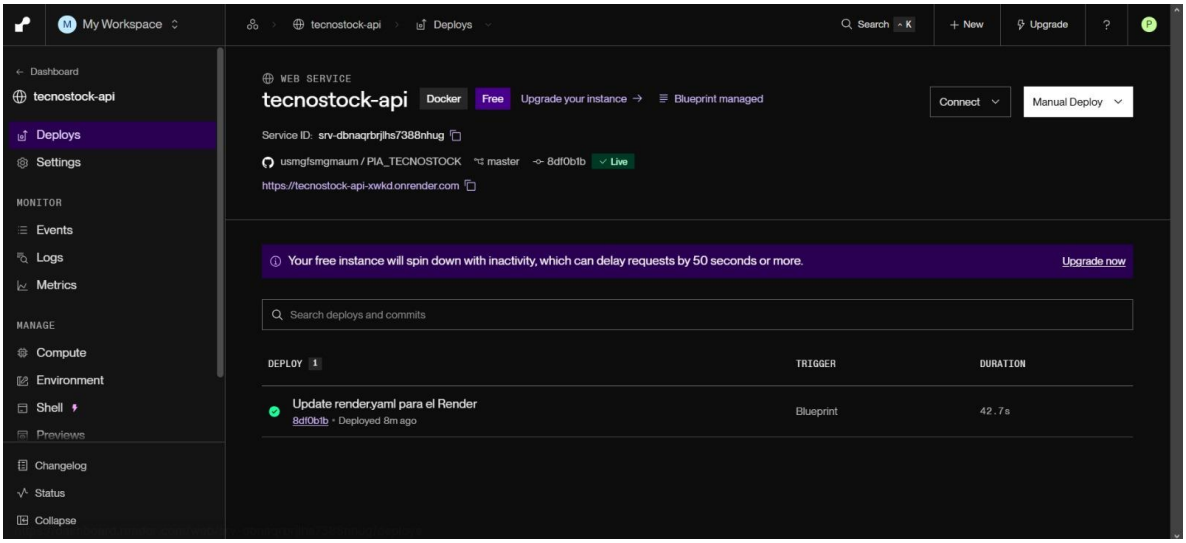
- Health check: https://tecnostock-api-xwkd.onrender.com/health
- Link Repositorio: usmgfsmgmaum/PIA_TECNOSTOCK: PIA TECNOSTOCK
- Aplicación: https://tecnostock-api-xwkd.onrender.com
- Swagger/OpenAPI: https://tecnostock-api-xwkd.onrender.com/docs

Reporte

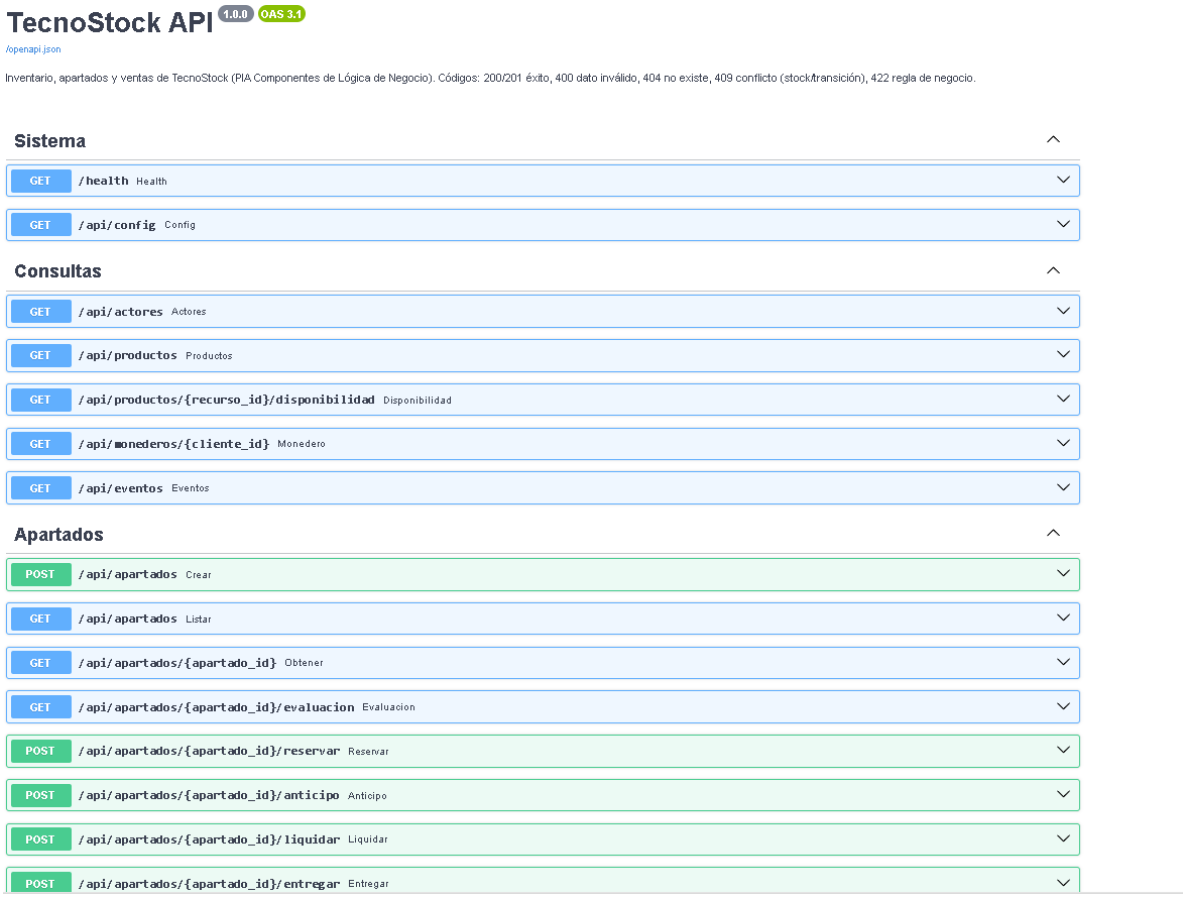

My Workspace
PIA_TECNOSTOCK
Resources
Search
New
Upgrade
Blueprints
PIA_TECNOSTOCK
Resources
Syncs
Settings
BLUEPRINT
PIA_TECNOSTOCK
Manual sync
Blueprint ID: exs-dbnaqe8u01pc73c7oqfg
usmgfsmgmaum / PIA_TECNOSTOCK
master
SERVICE NAME 2
STATUS
RUNTIME
REGION
UPDATED
tecnostock-api
Deployed
Docker
Oregon
6m ago
tecnostock-db
Available
PostgreSQL 18
Oregon
8m ago
Changelog
Status
Collapse


My Workspace
tecnostock-db
Info
Search
New
Upgrade
Dashboard
tecnostock-db
Info
MONITOR
Logs
Metrics
MANAGE
Plan
Apps
Recovery
POSTGRESQL
tecnostock-db
Free
Upgrade your instance
Blueprint managed
View docs
Connect
Service ID: dpg-dbnaqgrrjlhs7388mn40-a
Info
Your database will expire on December 6, 2026. The database will be deleted unless you upgrade to a paid compute plan.
General
Name
A unique name for your database.
tecnostock-db
Edit
Created
9 minutes ago
Status
Available
Changelog
Status
Collapse

Reporte



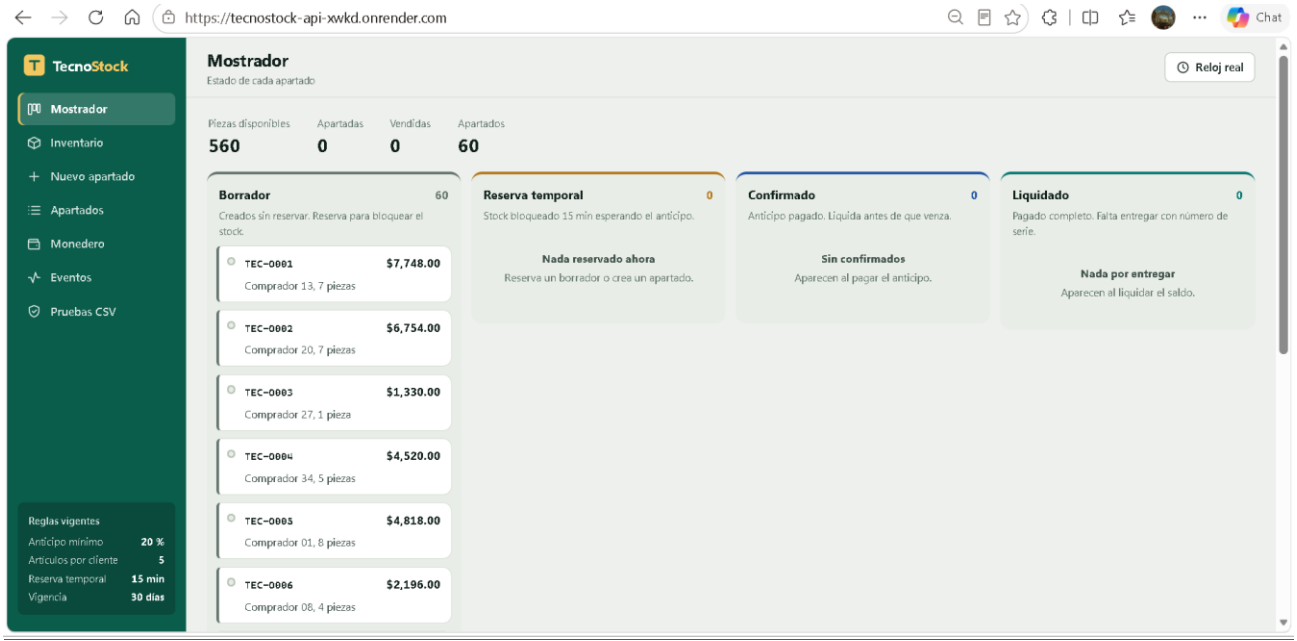
API en Render:



https://tecnostock-api-xwkd.onrender.com/docs

Aplicación en Render:

https://tecnostock-api-xwkd.onrender.com/



4.5 Seguridad de configuración y limitaciones

Las credenciales no están escritas en el código: llegan por variables de entorno y `.env.example` trae valores de ejemplo. Las contraseñas por omisión de Compose son solo para desarrollo local y habría que cambiarlas antes de exponer cualquier cosa. Estas son las limitaciones que conocemos y lo que haría falta para cerrarlas:

Reporte

Tema	Estado actual	Siguiente paso
Transferencias, venta directa y devoluciones	Diseñadas en la U3, sin código.	Implementar los agregados Transferencia, Venta y Devolución y sus endpoints.
Sucursales	Una sola sucursal (SUC-01), con la existencia del CSV.	Cargar las cinco sucursales para que US-02 sea completa.
Pagos	El monto se recibe como dato; no hay pasarela ni método de pago.	Definir catálogo de métodos y conectar una terminal o pasarela.
Autenticación y roles	La API no identifica al usuario; los roles del BPMN no se hacen cumplir.	Agregar identidad y control de acceso por rol.
Concurrencia (ADR-06)	Un lock por proceso serializa las escrituras.	Con varias réplicas, usar `SELECT ... FOR UPDATE` o versionado.
Vencimientos	Se procesan al llamar a un endpoint.	Programar un scheduler con reintentos y monitoreo.
Monedero	Solo recibe abonos; no se canjea.	Definir canje, vigencia y combinación con otros pagos.
Escaneo de QR	El ticket trae el código como texto.	Generar la imagen QR y leerla desde almacén.

4.6 Matriz de trazabilidad

Cada regla del BRD se puede seguir hasta el componente que la implementa y la prueba que la verifica. La versión editable vive en `docs/architecture/trazabilidad.md`.

BRD / historia	Regla	Componente	Prueba
BR1, US-01, US-02	Consulta de disponibilidad	`ConsultasService.disponibilidad_global`	`test_cu01_disponibilidad`
BR4, US-08	RN-01: anticipo del 20 %	`AnticipoMinimo`, `Apartado.confirmar_anticipo`	`test_anticipo_minimo`, `test_reserva_y_anticipo_reglas`
US-08, ADR-04	RN-02: máximo 5 por cliente	`MaximoArticulosPorCliente`, `contar_articulos_activos`	`test_limite_articulos`, `test_limite_5_global_por_cliente`
US-09	RN-03: reserva de 15 min	`Apartado.expirar_reserva`, handler `liberar_stock`	`test_us09_reserva_expirada_libera`, `test_pago_tardio_cancela_y_rechaza`
US-10	Confirmar; vence a 30 días	`Apartado.confirmar_anticipo`	`test_flujo_feliz`
US-11	RN-05: liquidar y emitir QR	`Apartado.liquidar`	`test_liquidar_reglas`, `test_flujo_completo_y_salida_fisica`
US-12	RN-04: vencimiento y monedero	`Apartado.vencer`, `Monedero.abonar_anticipo`	`test_us12_vencimiento_monedero_idempotente`
BRD 5.2, US-06	RN-07: nunca vender lo apartado	`Inventario.apartar`	`test_no_se_vende_lo_apartado`, `test_apartar_liberar_vender`
US-14	RN-08: falla, rollback	`UnitOfWork`	`test_rechazos_de_creacion`, `test_reservar_borrador_y_rollback_uow`
US-15, US-16, BR3	RN-06: serie única y salida única	`Apartado.entregar`, `Inventario.descontar_por_venta`	`test_entrega_series`, `test_flujo_completo_y_salida_fisica`
Modelo del PIA	Specifications AND/OR/NOT	`puede_procesarse()`, `beneficio_referencia()`	`TestSpecifications`, `test_runner_csv`
BRD 5.2	State Machine	`MAQUINA_APARTADO`	`test_transiciones_invalidas`
Contrato de la API	Códigos 201, 400, 404, 409, 422	`src/api/main.py`	`tests/test_api.py`

Link del GitHub

Repositorio con el código y toda la documentación:
https://github.com/usmgfsmgmaum/PIA_TECNOSTOCK

Conclusión general

El desarrollo de TecnoStock nos hizo ver que antes de programar hay que entender bien el negocio. Pasamos de los requerimientos y las reglas de las unidades anteriores a un prototipo que se puede ejecutar, probar y demostrar, y en el camino se notó cuánto ayuda tener las reglas escritas: cuando una regla cambia, se sabe qué clase y qué prueba tocar.

Con la aplicación logramos representar el proceso central de la empresa: el apartado de productos, desde la reserva temporal hasta la entrega con número de serie, incluyendo la cancelación y el vencimiento con abono al monedero. Domain-Driven Design, Specification y State Machine nos ayudaron a ordenar la lógica y a impedir transiciones que no tienen sentido, como liquidar un apartado vencido.

También aprendimos a ser claros con el alcance. El diseño de la U3 incluye transferencias, ventas directas y devoluciones que no alcanzamos a programar, y preferimos documentarlo en lugar de dar a entender que existen. Lo que sí está en el repositorio tiene pruebas, cobertura de 98.37 % y una forma reproducible de ejecutarse con Docker Compose y GitHub Actions. TecnoStock queda como una base sólida para seguir creciendo en futuras versiones.

Conclusiones individuales

Rodrigo Noguera Pérez (2136238)

Durante la realización de este proyecto aprendí que el análisis de los requerimientos es una de las etapas más importantes del desarrollo de software, ya que permite comprender qué necesita realmente una empresa y cómo se deben representar sus procesos dentro de un sistema. Trabajar con TecnoStock me permitió entender mejor cómo convertir las reglas del negocio en funcionalidades y cómo utilizar conceptos como DDD, entidades, agregados y patrones de diseño para organizar una aplicación. También comprendí la importancia de mantener una buena estructura en el código para facilitar su mantenimiento y futuras modificaciones. Considero que este proyecto me ayudó a fortalecer mis conocimientos de programación y a comprender mejor cómo se desarrolla una solución de software desde su planeación hasta su implementación.

Andrik Hasiel Pérez Limon (2118898)

La elaboración de este proyecto me permitió conocer la importancia de desarrollar un sistema tomando en cuenta las necesidades y reglas específicas de una empresa. Uno de los

aprendizajes más importantes fue comprender que cada proceso debe tener condiciones bien definidas para evitar errores y mantener un funcionamiento adecuado. Además, pude reforzar mis conocimientos sobre historias de usuario, criterios de aceptación, diagramas de procesos y arquitectura de software. También comprendí la utilidad de realizar pruebas para verificar que las funcionalidades cumplan con lo establecido durante el análisis. En general, esta actividad me permitió desarrollar una visión más completa sobre el trabajo que existe detrás de una aplicación y la importancia de documentar correctamente cada una de sus etapas.

Erick Antonio Paez Estrada (2120761)

A través del desarrollo de TecnoStock pude comprender mejor cómo se relacionan los diferentes componentes que forman parte de una aplicación de software. Este proyecto me permitió reforzar mis conocimientos sobre programación, bases de datos, servicios API y organización de proyectos, además de conocer la utilidad de herramientas como Docker y GitHub Actions dentro del desarrollo de software. También aprendí que implementar una funcionalidad no es suficiente si no se comprueba que trabaja correctamente y respeta las reglas establecidas por el negocio. Considero que uno de los mayores aprendizajes fue entender cómo los conocimientos teóricos de la materia pueden aplicarse para resolver una problemática empresarial mediante una solución tecnológica. Finalmente, este trabajo me ayudó a reconocer la importancia de la colaboración y de mantener una buena organización durante el desarrollo de un proyecto en equipo.